



# HUST

**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



# APPLIED ALGORITHMS



ĐẠI HỌC  
BÁCH KHOA HÀ NỘI  
HANOI UNIVERSITY  
OF SCIENCE AND TECHNOLOGY

# APPLIED ALGORITHMS

## DEPTH FIRST SEARCH (DFS) AND APPLICATIONS

ONE LOVE. ONE FUTURE.

- **Depth First Search (DFS)**
- DFS tree and và structure Num, Low
- Find bridges (cạnh cầu)
- Find articulation vertex (đỉnh khớp)
- Find strongly connected component (thành phần liên thông mạnh)

# Depth First Search (Tìm kiếm theo chiều sâu)

- Depth First Search is a basic graph traversal technique (visiting every vertex and every edge of the graph).
  - The algorithm can answer the question, does there exist a path from vertex  $u$  to vertex  $v$  on graph  $G$  or not, if yes, indicate it.
  - The algorithm not only answers whether there is a path from  $u$  to  $v$ , but it can answer which other vertices on the graph  $G$  can be reached from  $u$ .
- The traversal order in DFS follows the ***Last In First Out*** (LIFO) mechanism, and starts from a certain starting vertex  $u$ .
  - Can use backtracking or stack to implement
- The complexity:  $O(|V| + |E|)$ , where  $V$  is the vertex set and  $E$  is the edge set of the graph  $G$ , as each vertex and edge of  $G$  is visited exactly once.

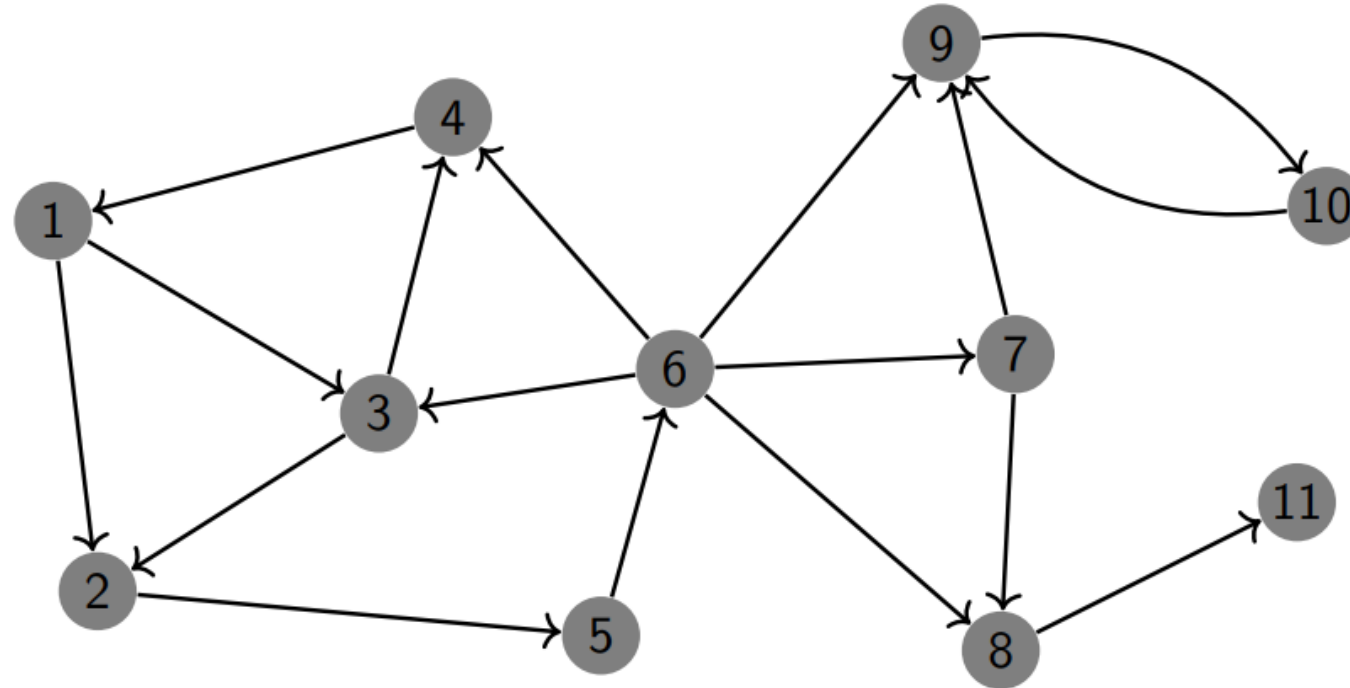
# Implement idea

```
//Main Program
for  $s \in V$ 
    visited[s]  $\leftarrow$  false
    for  $s \in V$ 
        if (visited[s] == false)
            DFS(s)
```

```
//DFS using recursive approach
DFS(s)
    visited[s]  $\leftarrow$  true // Visist s
    for each  $v \in \text{Adj}[s]$ 
        if (visited[v] == false)
            DFS(v)
```

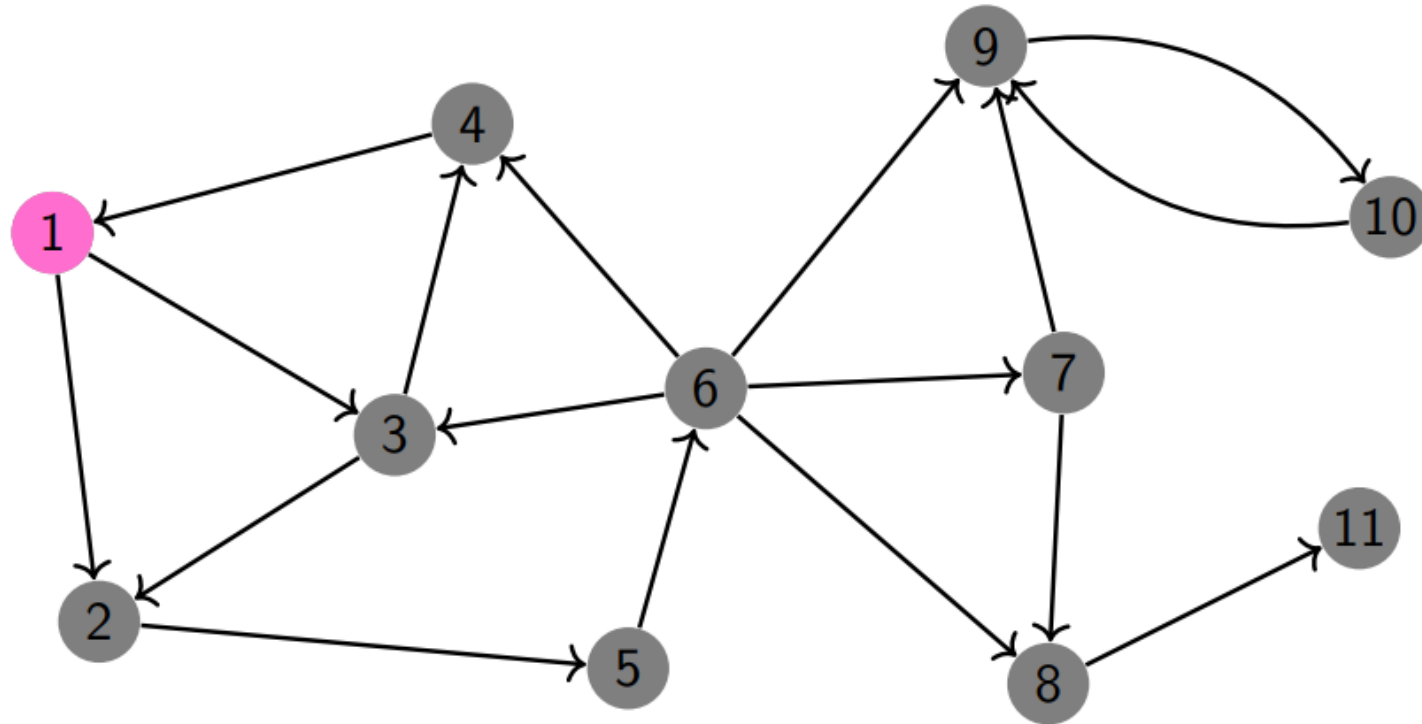
```
//DFS using Stack
DFS(s)
    S.push(s) // Insert s into the
    stack S
    While (S is not empty)
         $u = S.top()$ 
        S.pop()
        visited[u]  $\leftarrow$  true // Visist u
        for each  $v \in \text{Adj}[u]$ 
            if (visited[v] == false)
                S.push(v)
```

# Example



|         |  |   |   |   |   |   |   |   |   |   |    |    |
|---------|--|---|---|---|---|---|---|---|---|---|----|----|
| Stack:  |  |   |   |   |   |   |   |   |   |   |    |    |
|         |  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited |  | F | F | F | F | F | F | F | F | F | F  | F  |

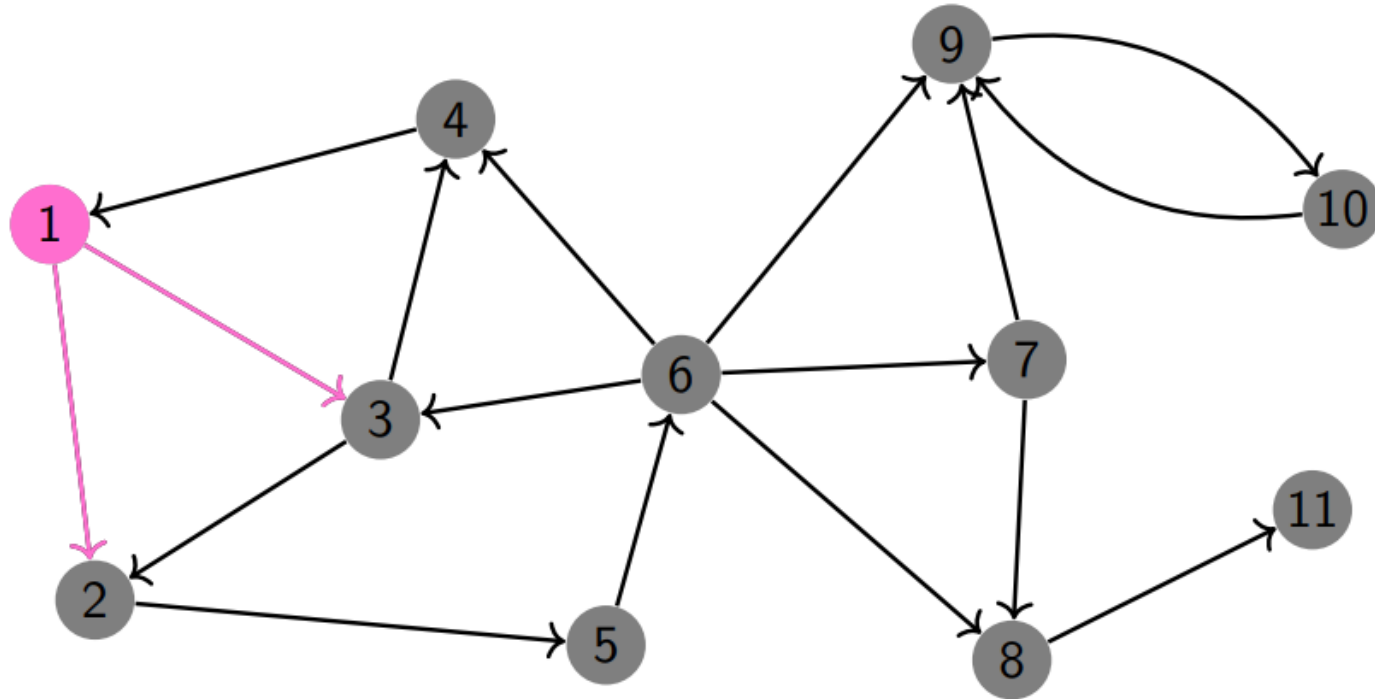
# Example



Stack: 1 |

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | F | F | F | F | F | F | F | F | F  | F  |

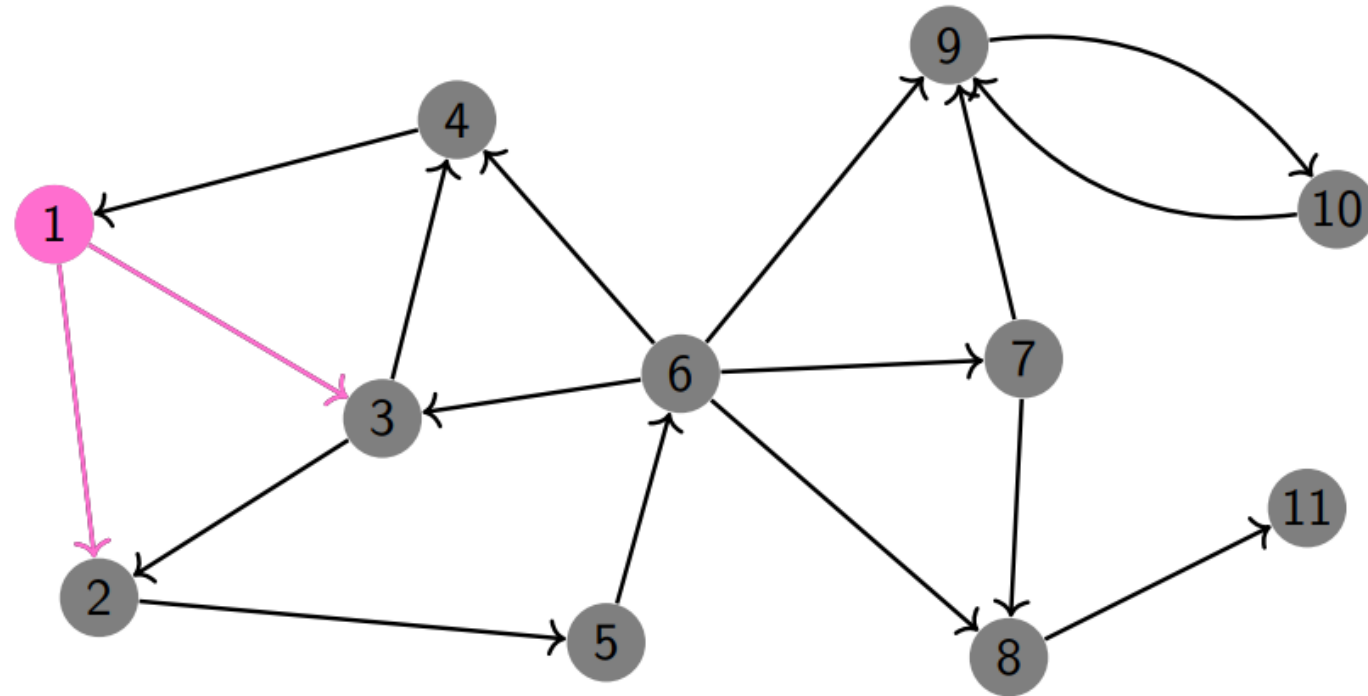
# Example



Stack: 1 |

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | F | F | F | F | F | F | F | F | F  | F  |

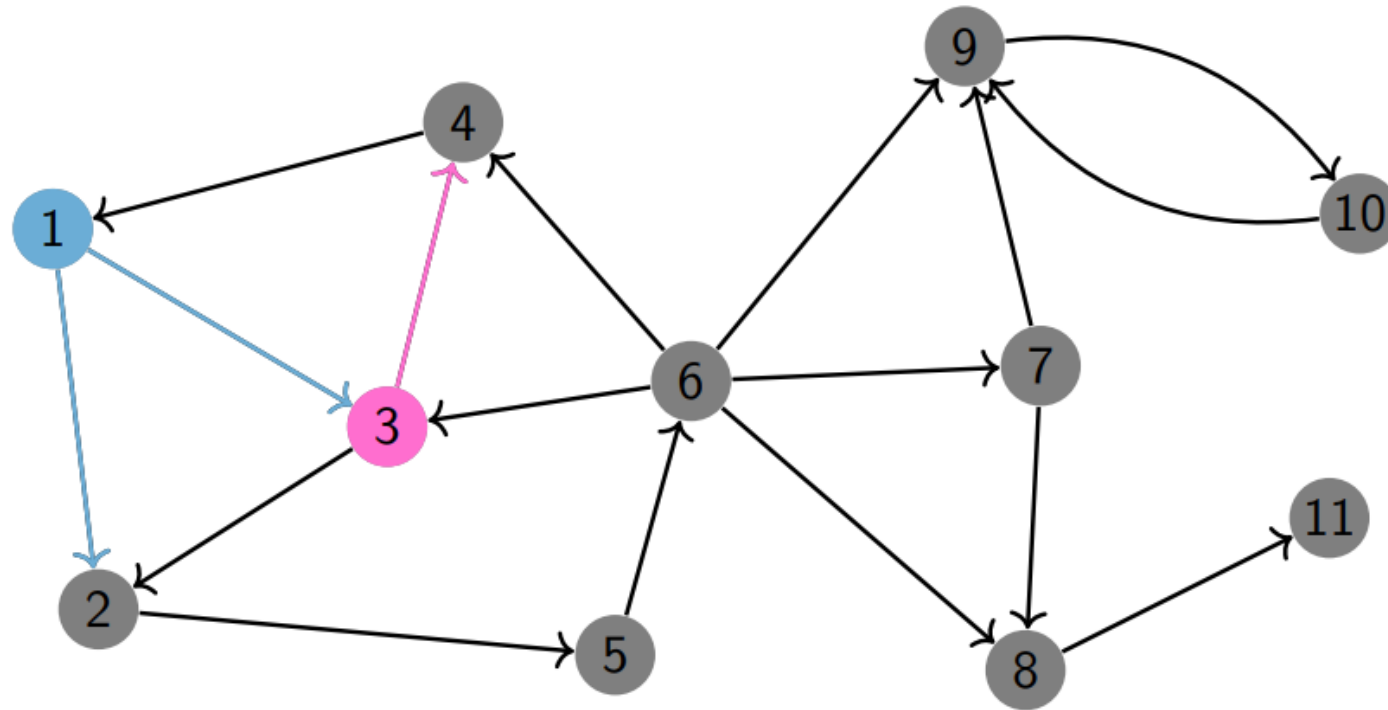
# Example



Stack: 1 | 3 2

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | F | F | F | F | F | F | F  | F  |

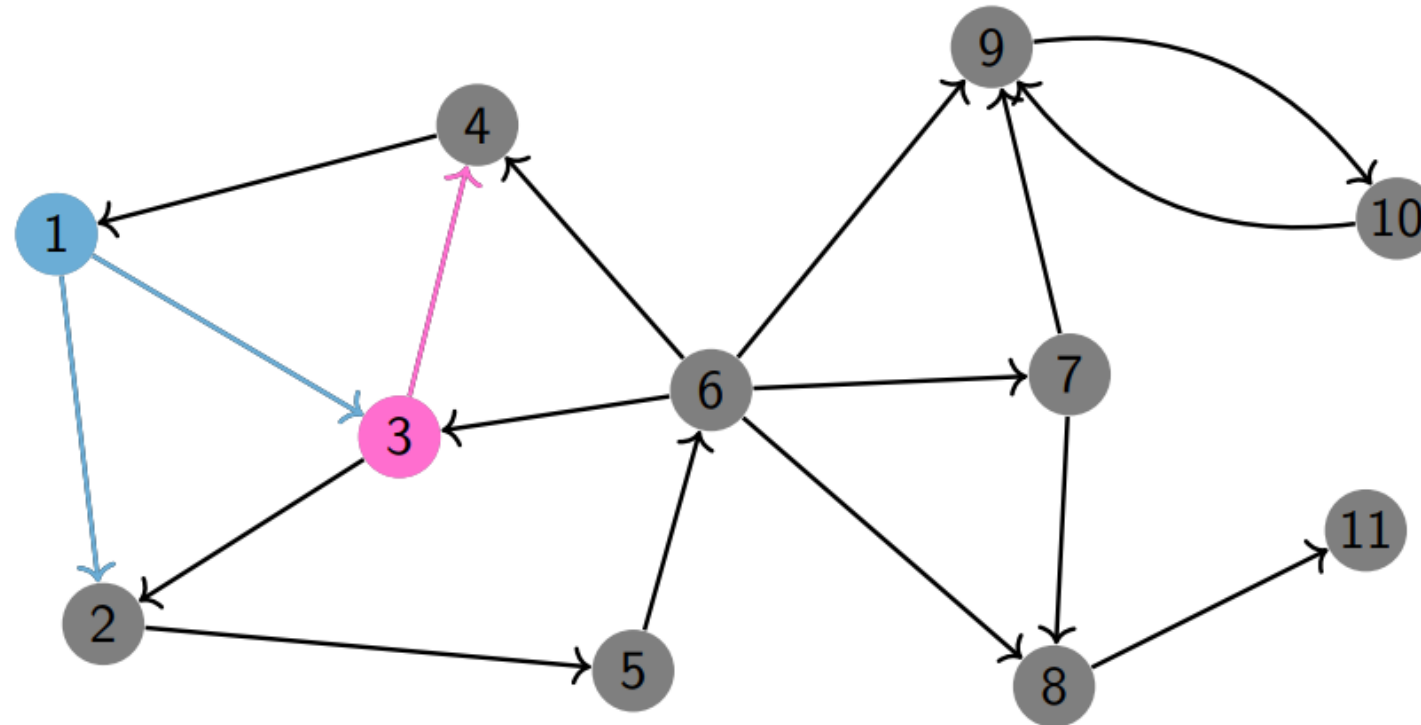
# Example



Stack: 3 | 2

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | F | F | F | F | F | F | F  | F  |

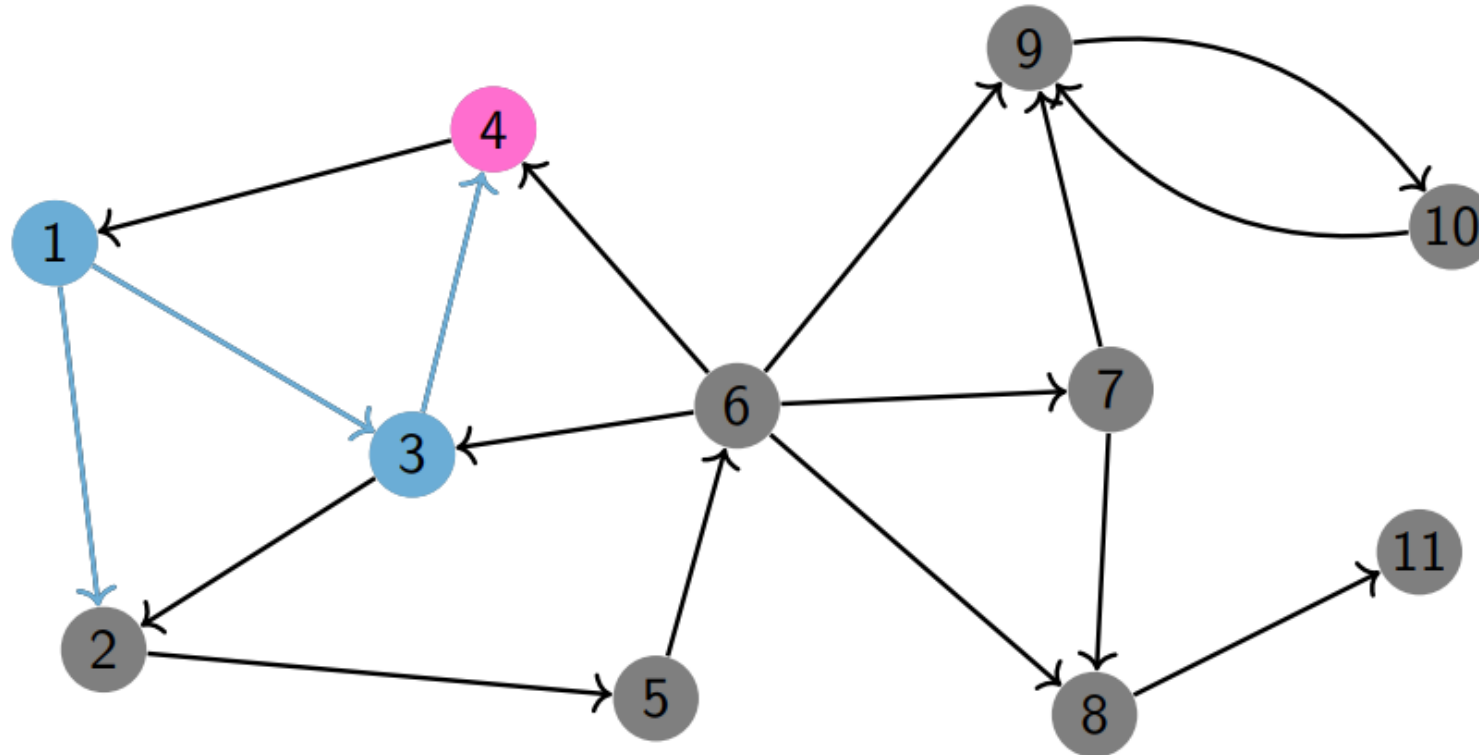
# Example



Stack: 3 | 4 2

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | F | F | F | F | F | F  | F  |

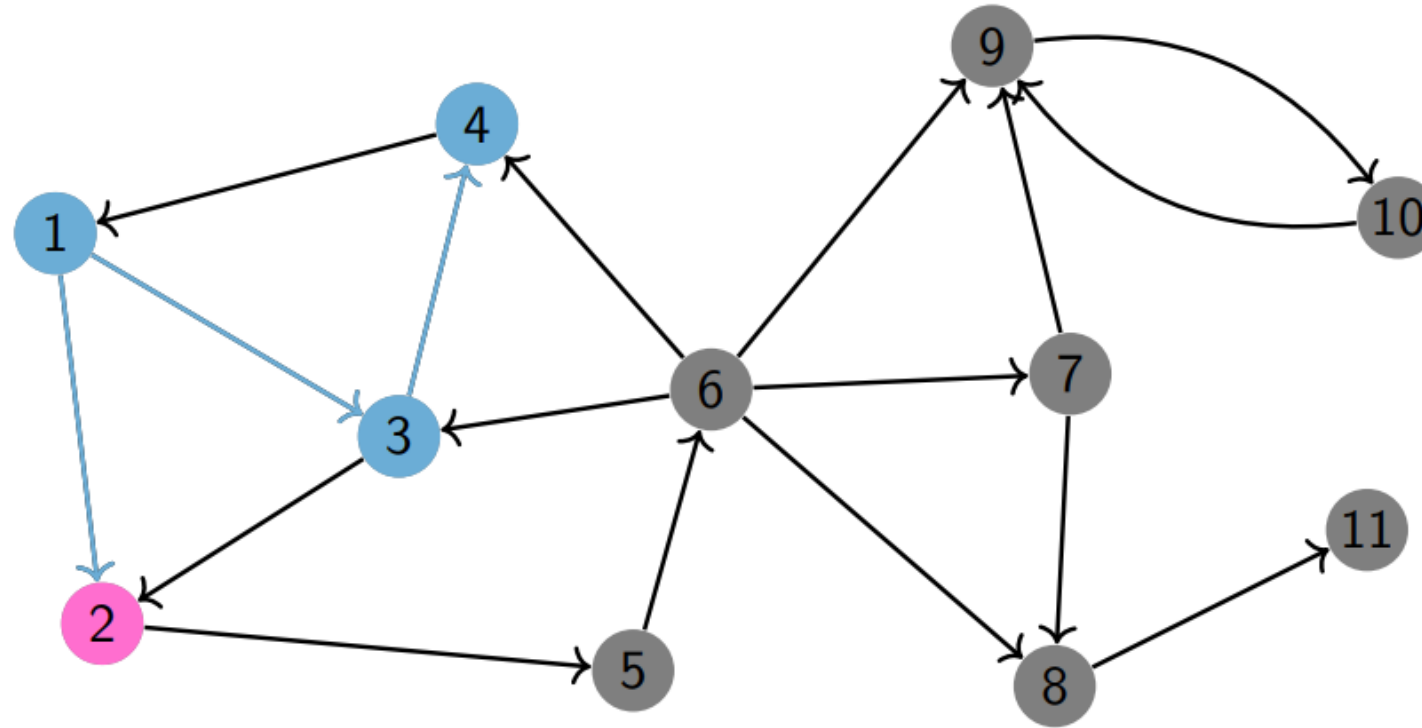
# Example



Stack: 4 | 2

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | F | F | F | F | F | F  | F  |

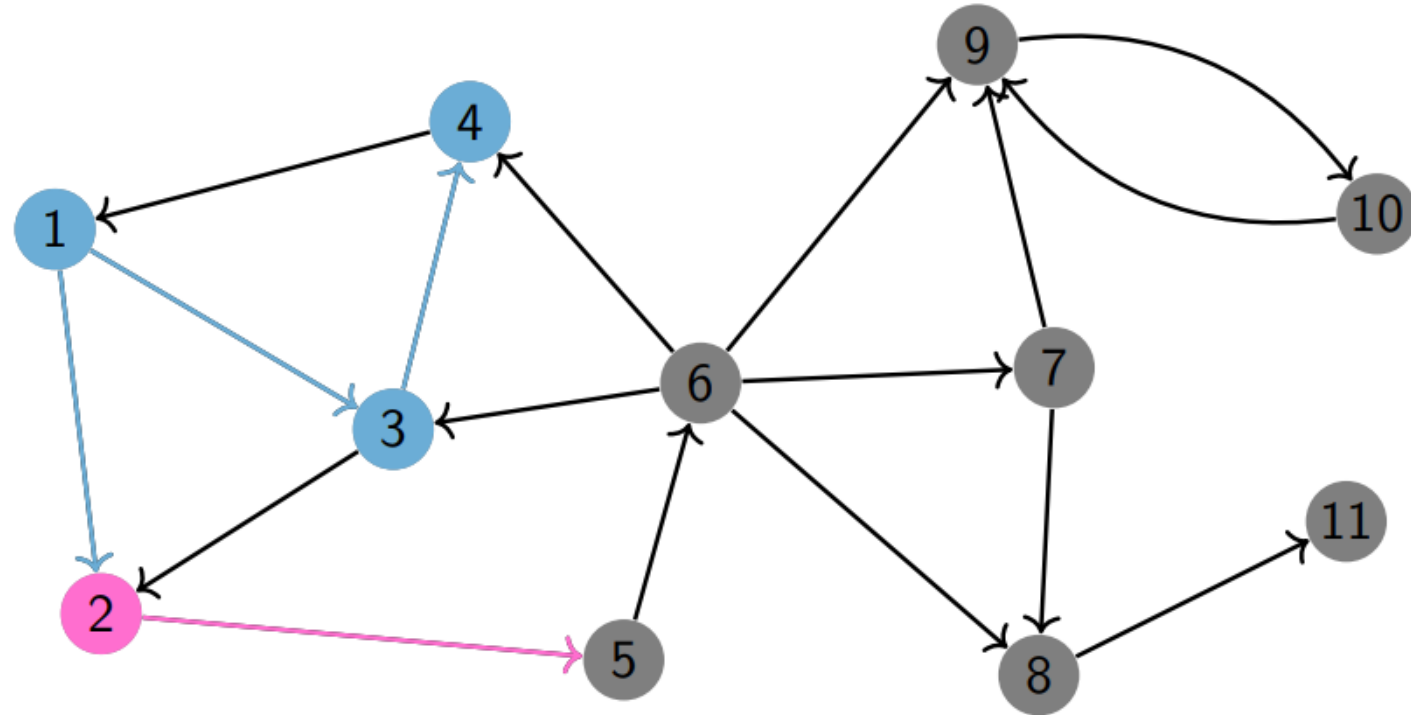
# Example



Stack: 2 |

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | F | F | F | F | F | F  | F  |

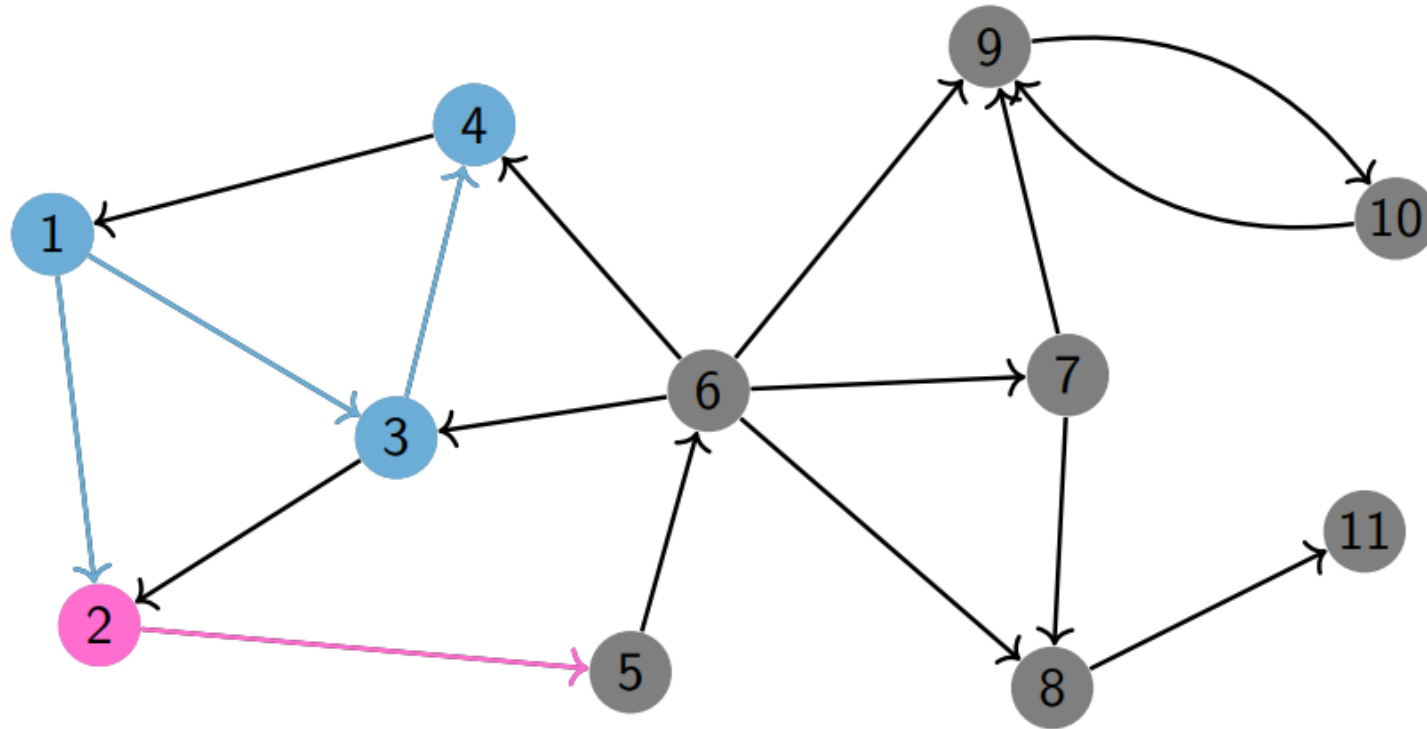
# Example



Stack: 2 |

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | F | F | F | F | F | F  | F  |

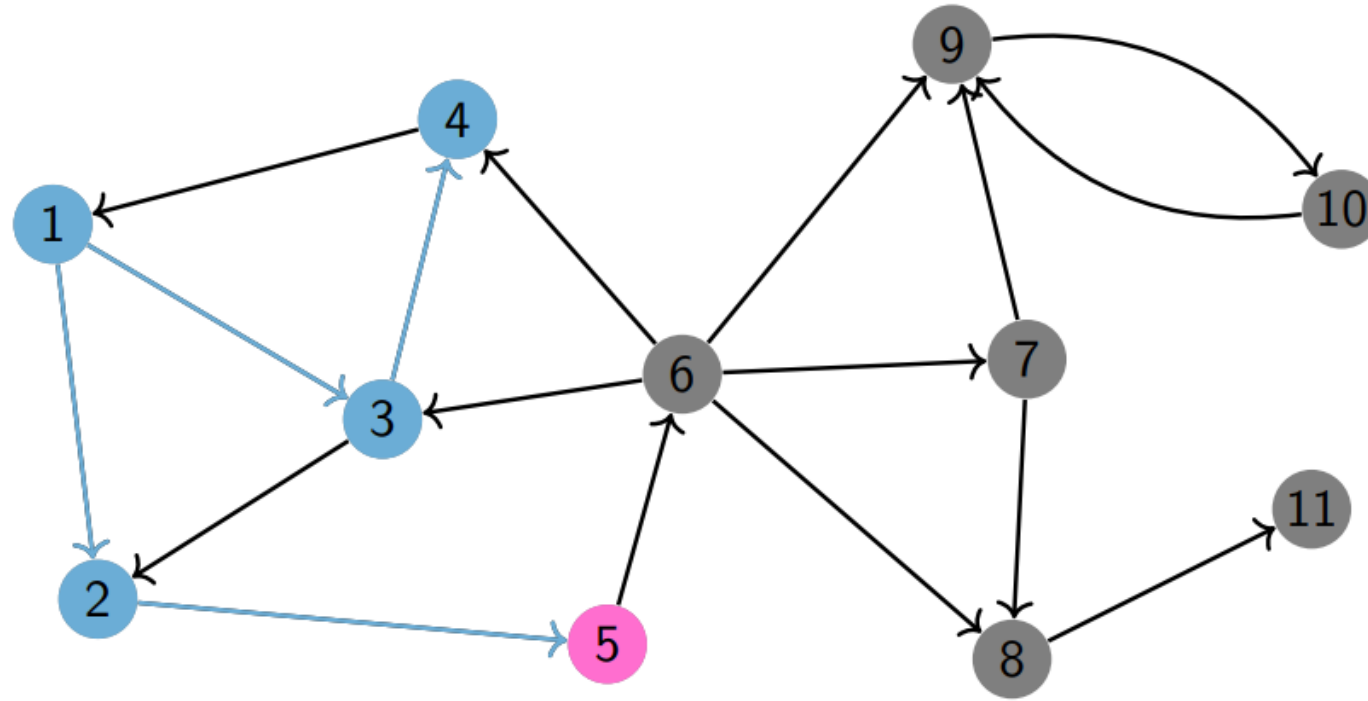
# Example



Stack: 2 | 5

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | F | F | F | F | F  | F  |

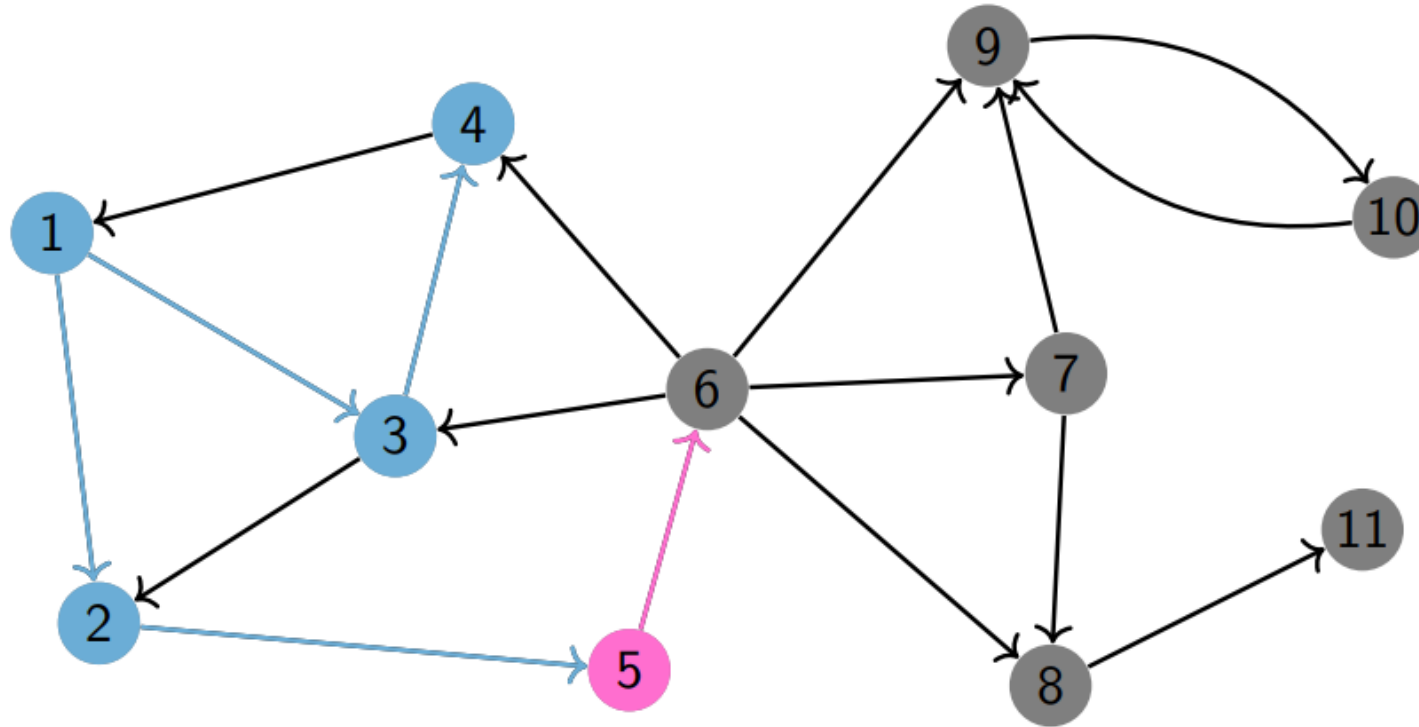
# Example



Stack: 5 |

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | F | F | F | F | F  | F  |

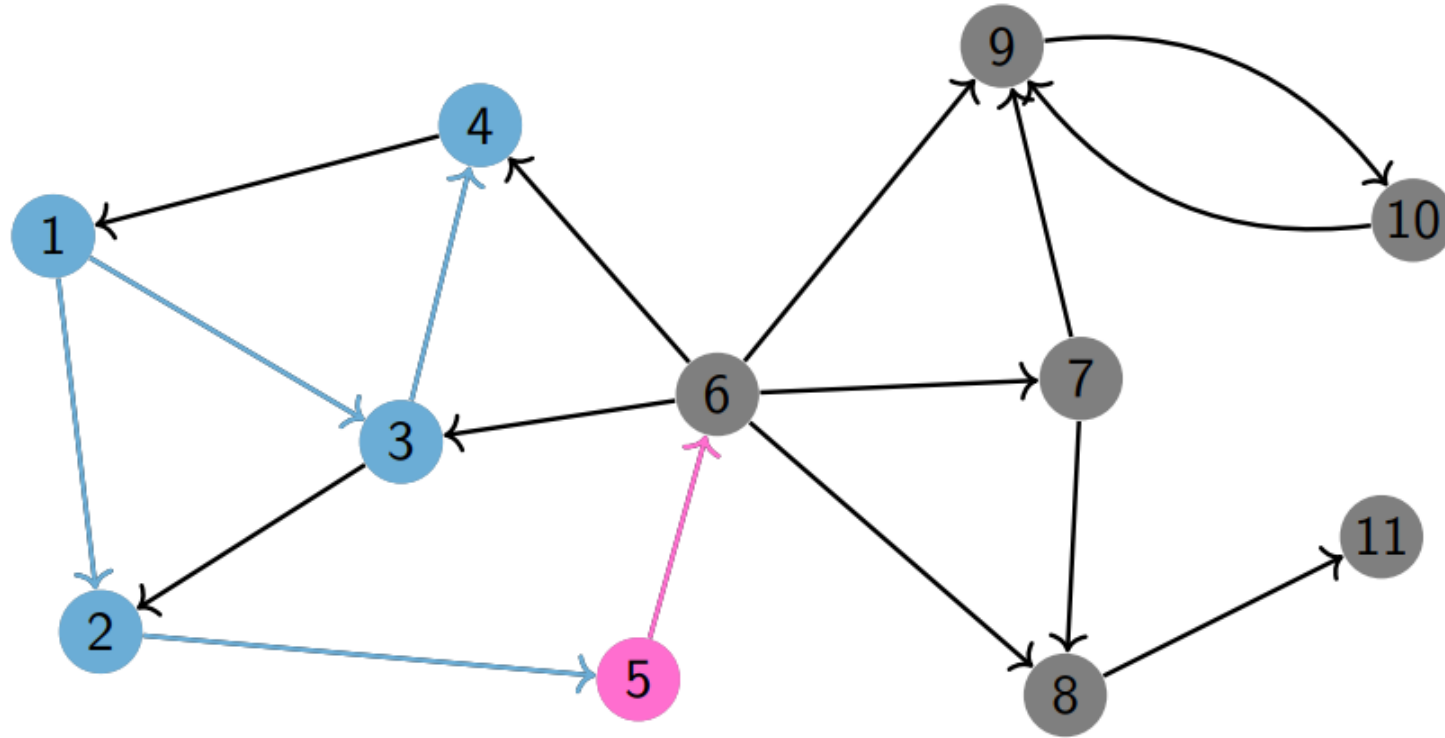
# Example



Stack: 5 |

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | F | F | F | F | F  | F  |

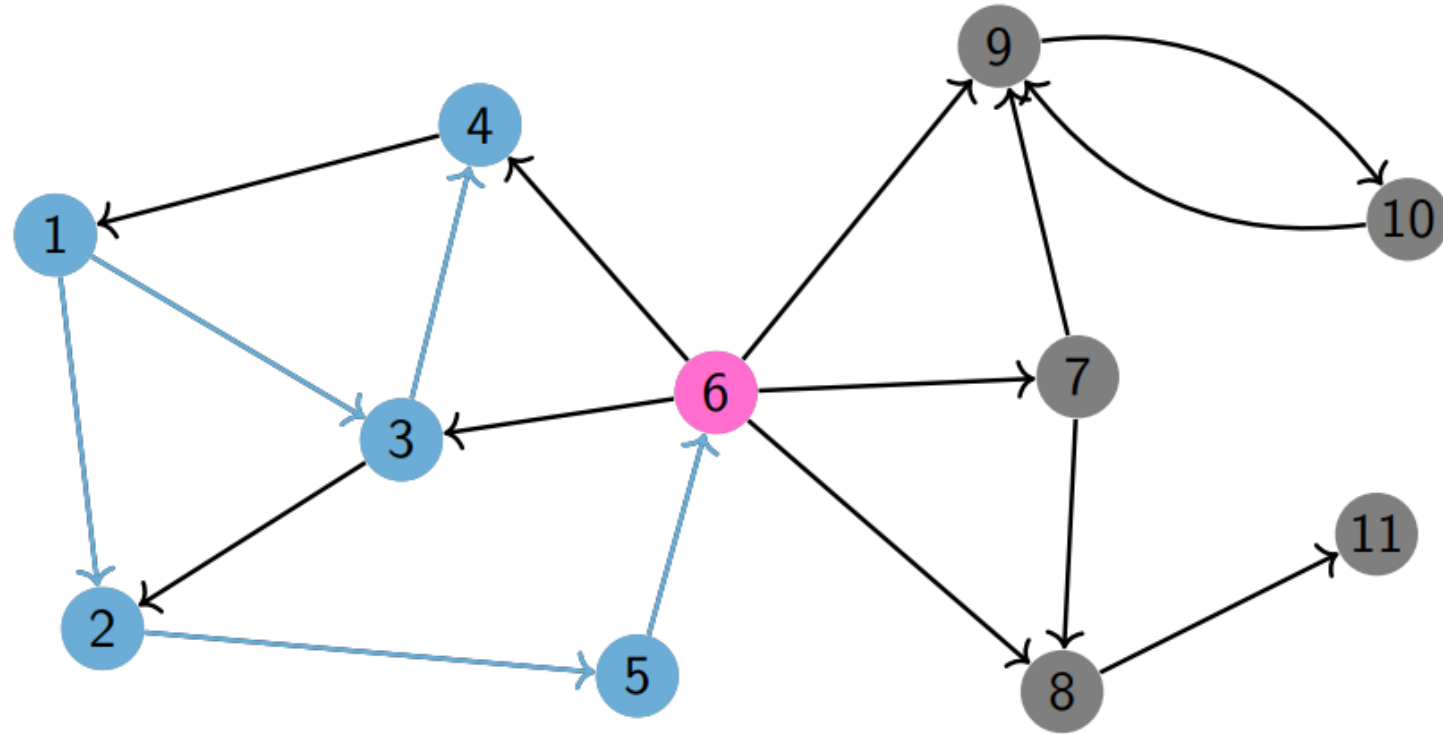
# Example



Stack: 5 | 6

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | T | F | F | F | F  | F  |

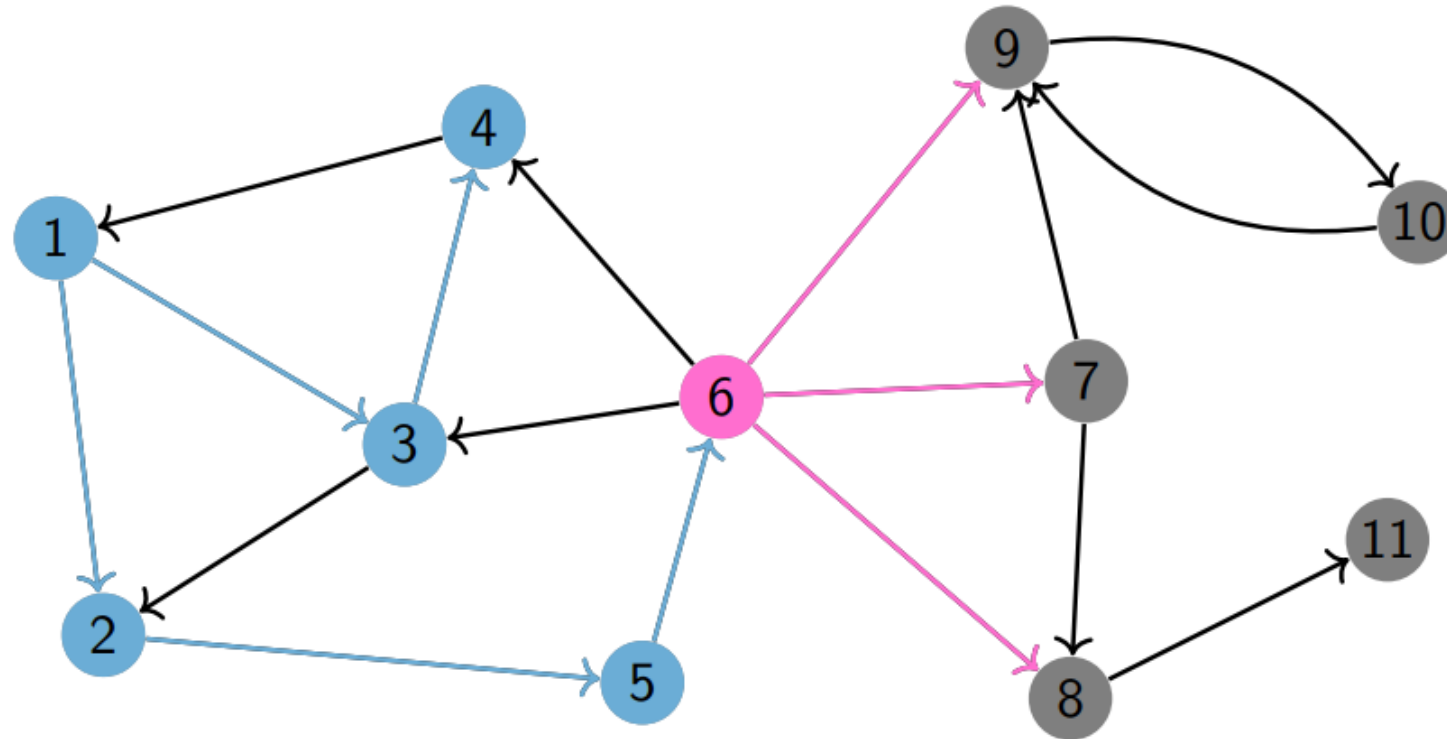
# Example



Stack: 6 |

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | T | F | F | F | F  | F  |

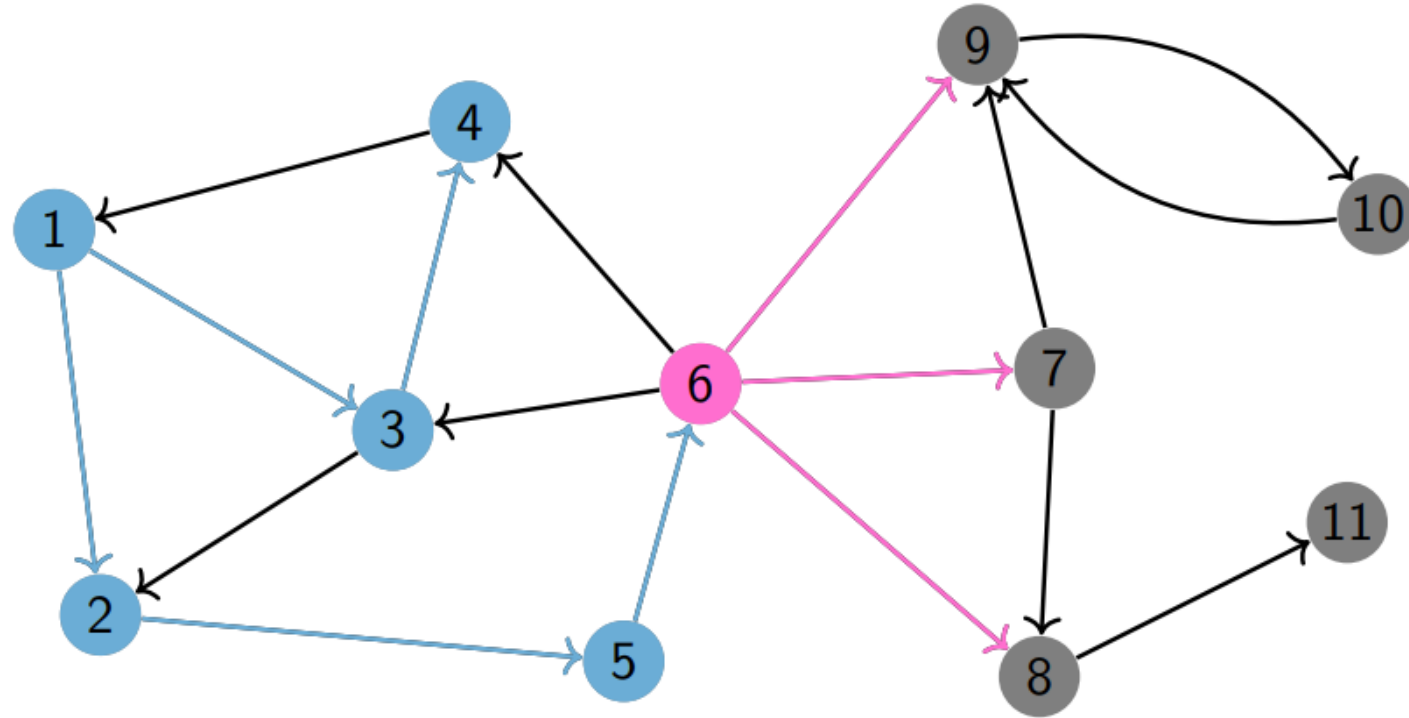
# Example



Stack: 6 |

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | T | F | F | F | F  | F  |

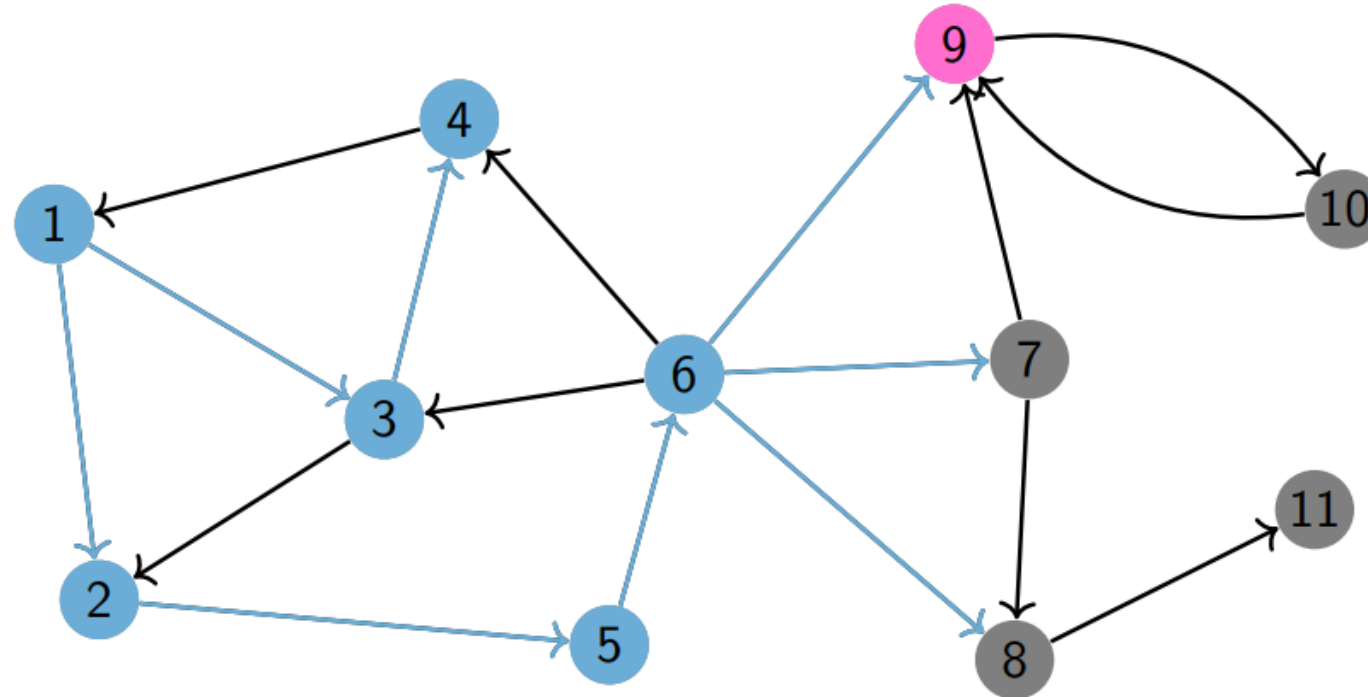
# Example



Stack: 6 | 9 7 8

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | T | T | T | T | F  | F  |

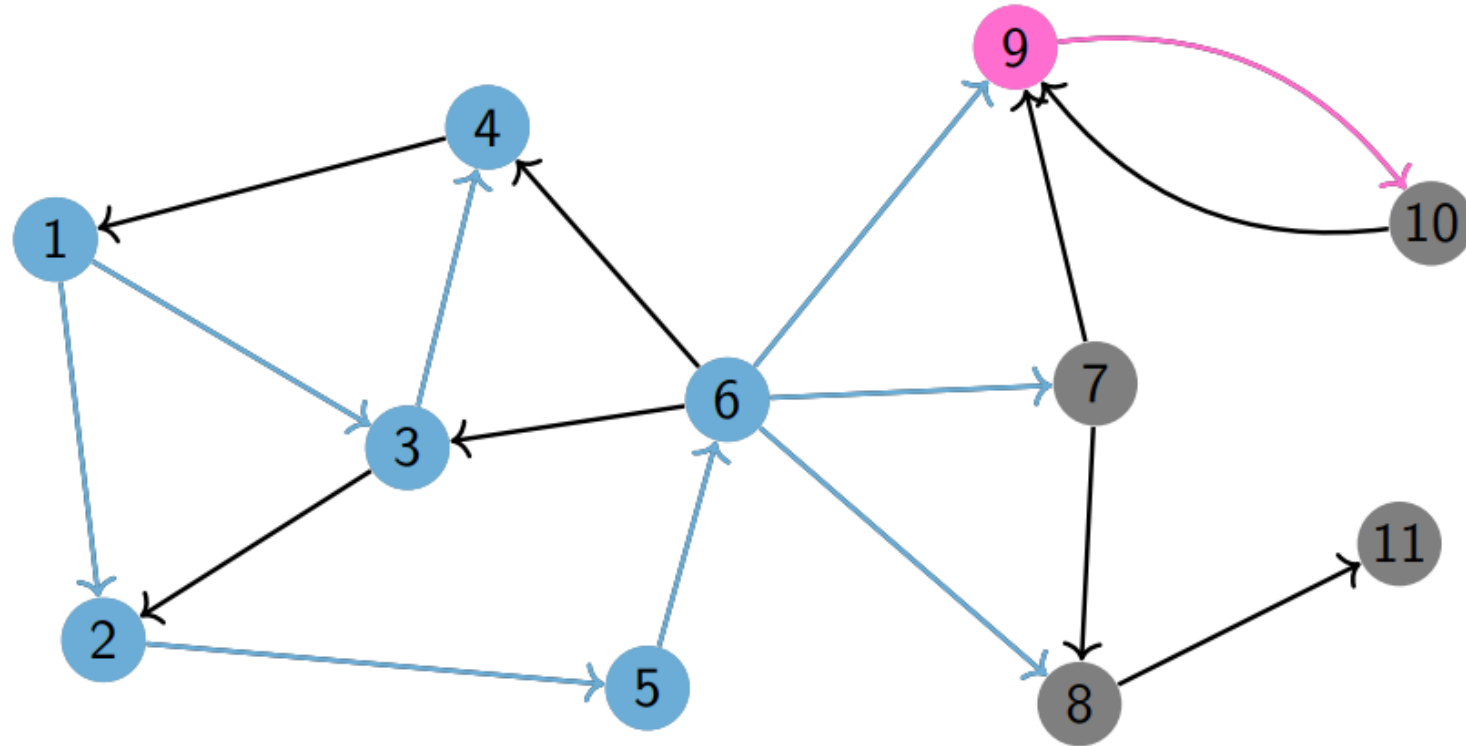
# Example



Stack: 9 | 7 8

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | T | T | T | T | F  | F  |

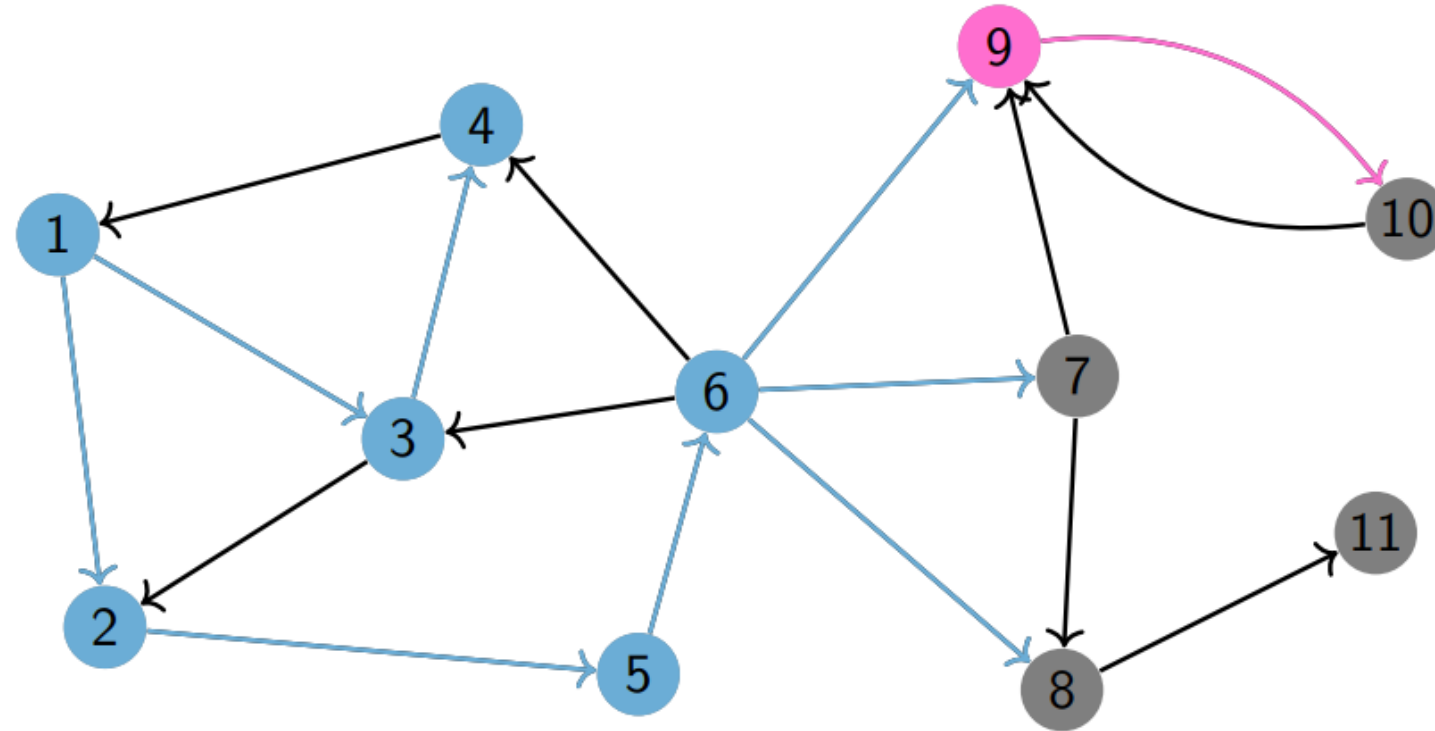
# Example



Stack: 9 | 7 8

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | T | T | T | T | F  | F  |

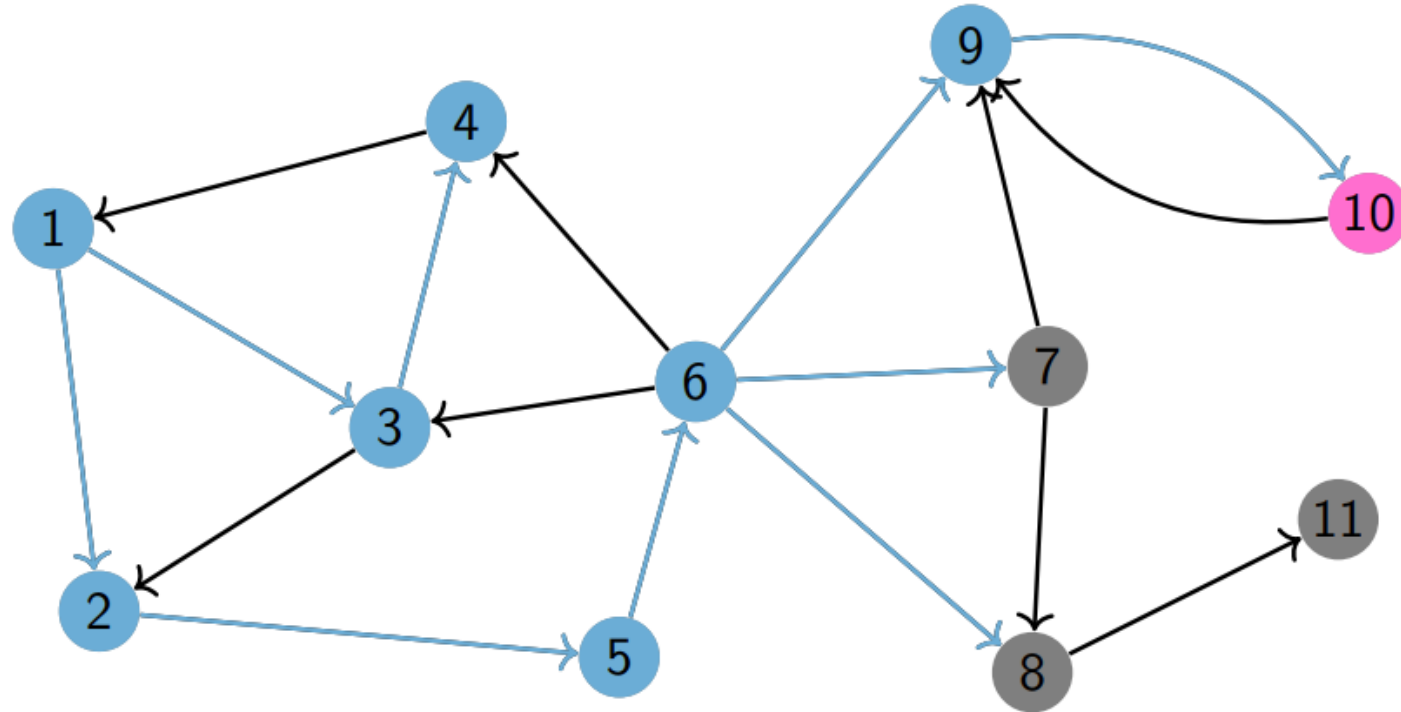
# Example



Stack: 9 | 10 7 8

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | T | T | T | T | T  | F  |

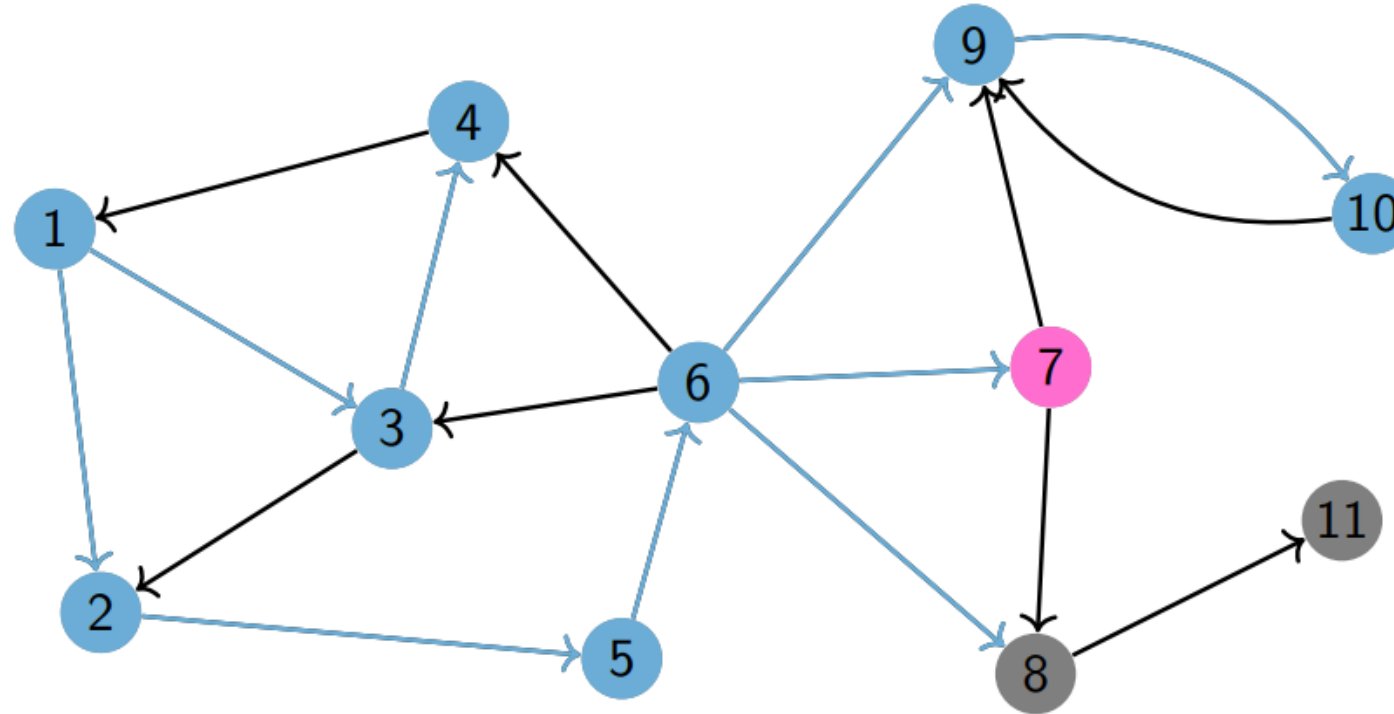
# Example



Stack: 10 | 7 8

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | T | T | T | T | T  | F  |

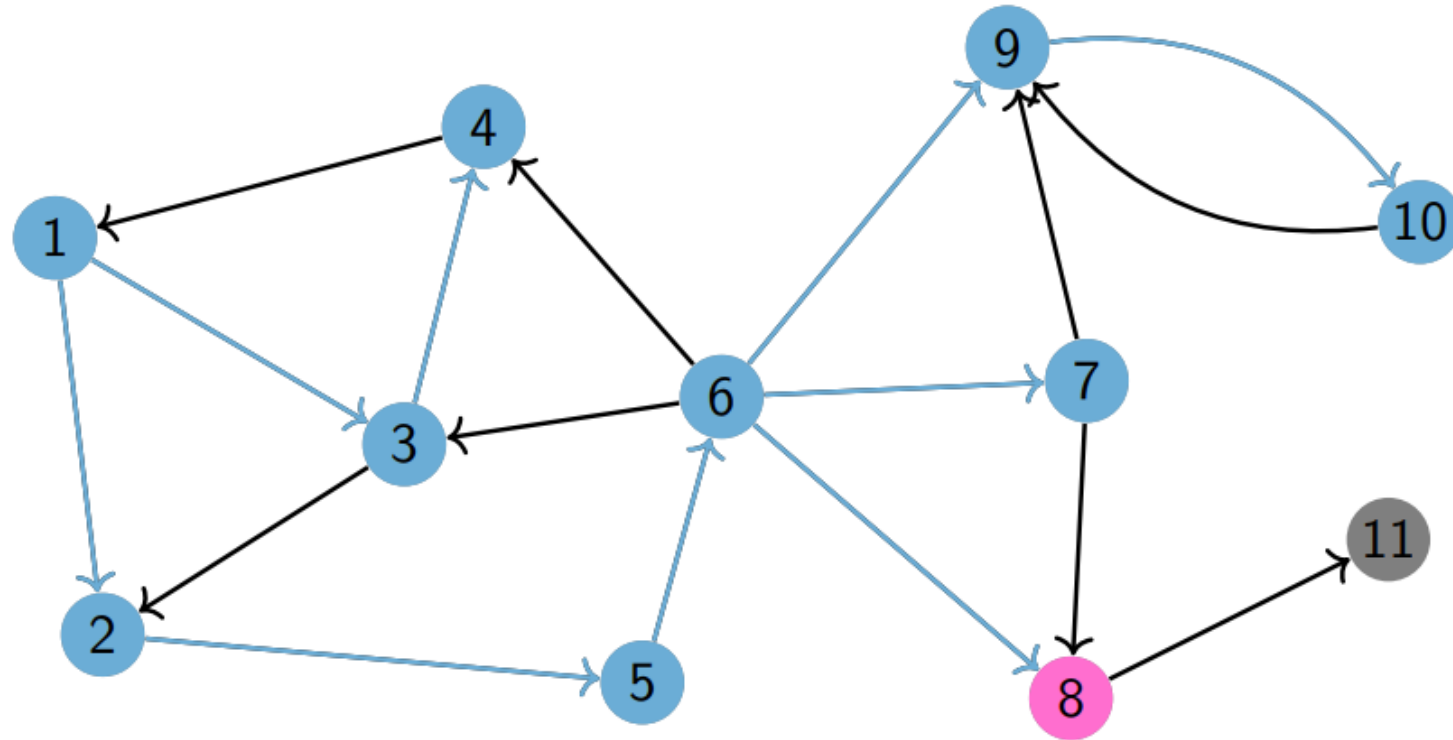
# Example



Stack: 7 | 8

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | T | T | T | T | T  | F  |

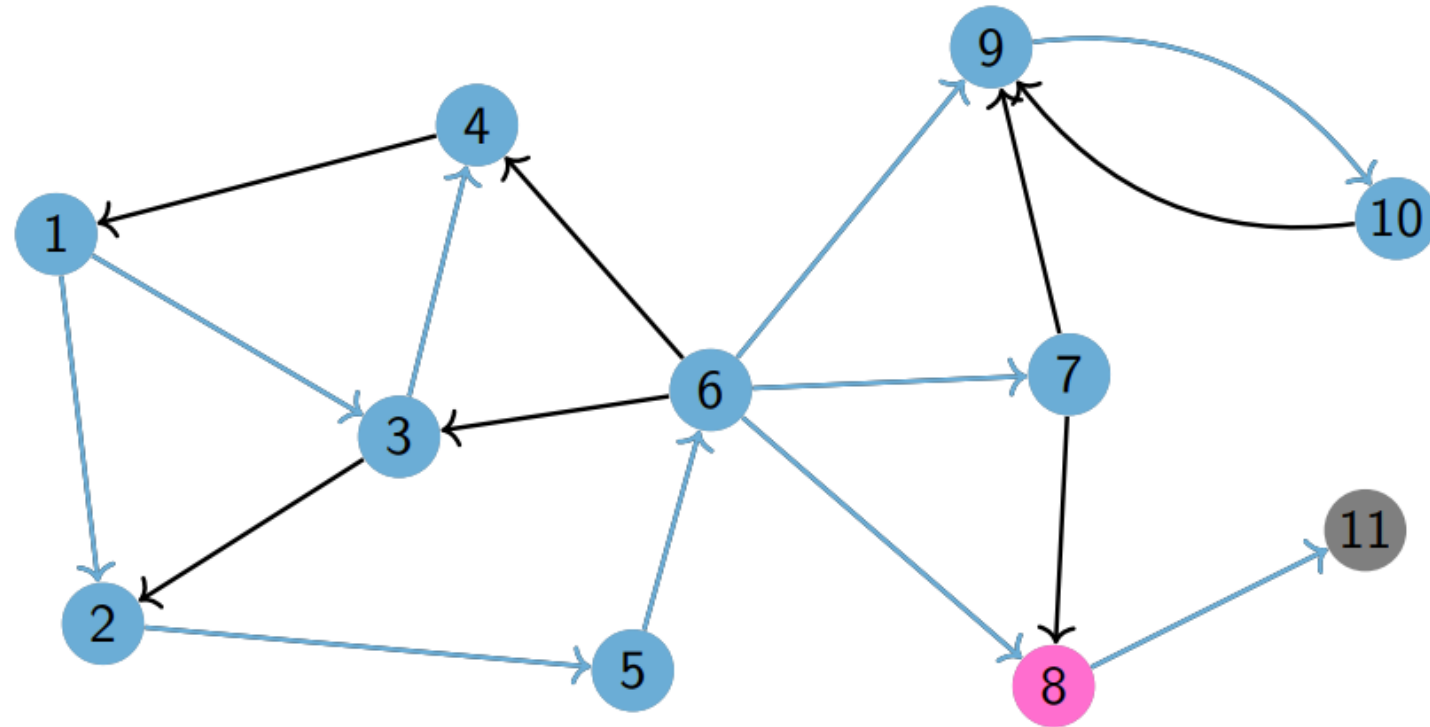
# Example



Stack: 8 |

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | T | T | T | T | T  | F  |

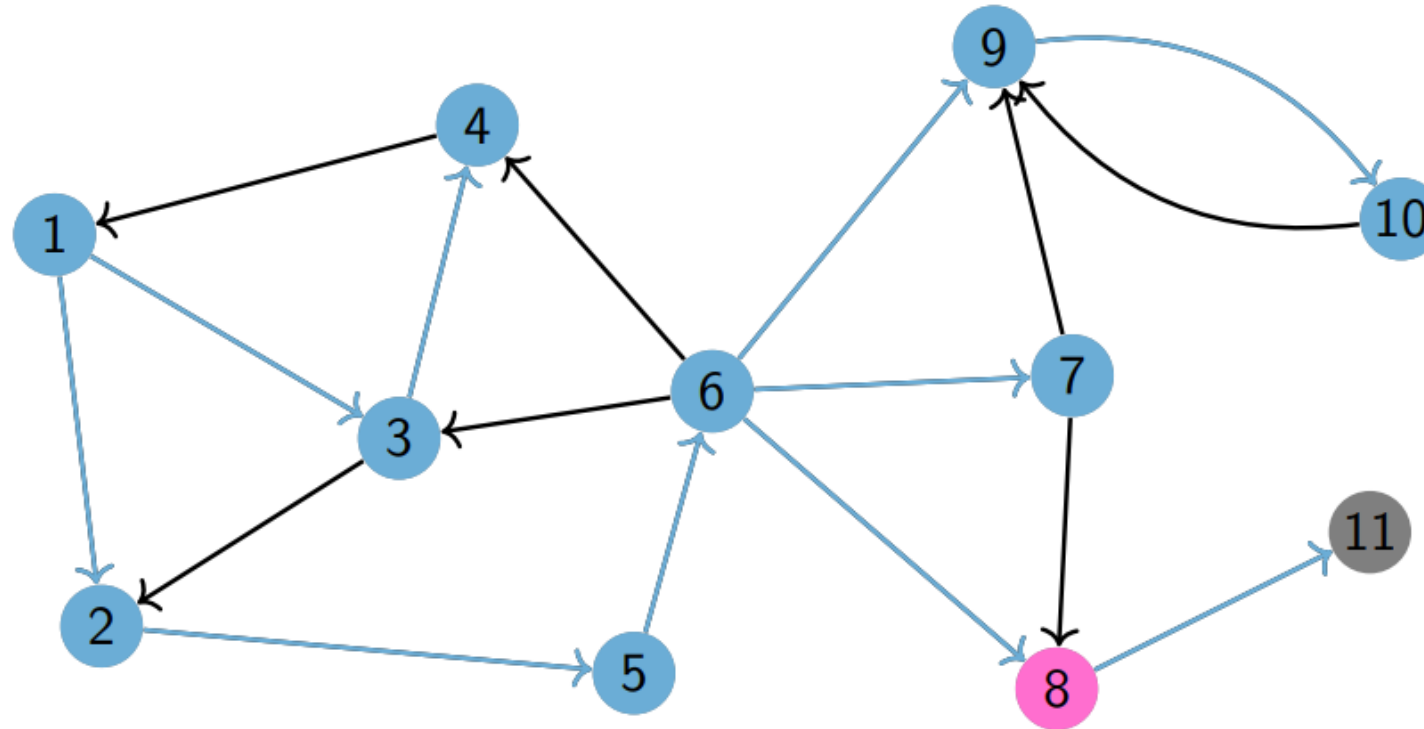
# Example



Stack: 8 |

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | T | T | T | T | T  | F  |

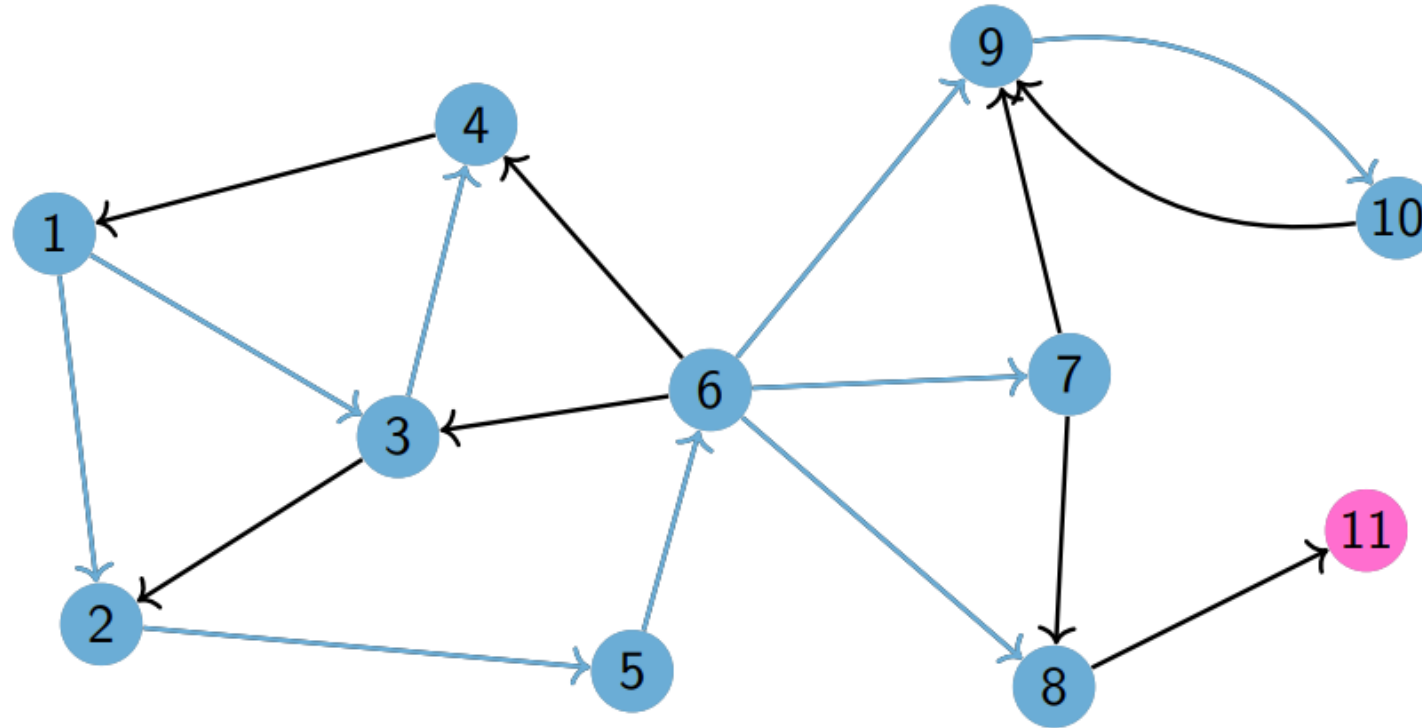
# Example



Stack: 8 | 11

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | T | T | T | T | T  | T  |

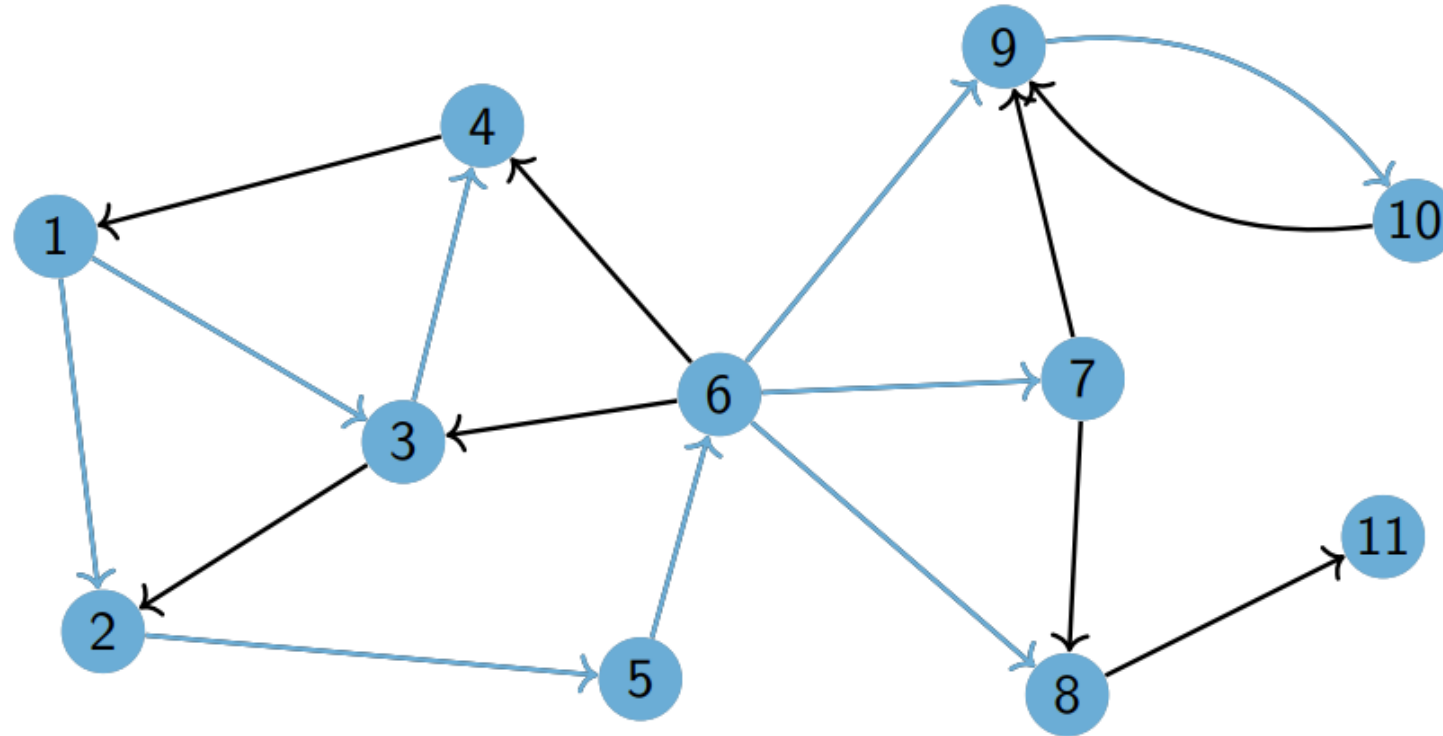
# Example



Stack: 11 |

|         |   |   |   |   |   |   |   |   |   |    |    |
|---------|---|---|---|---|---|---|---|---|---|----|----|
|         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited | T | T | T | T | T | T | T | T | T | T  | T  |

# Example

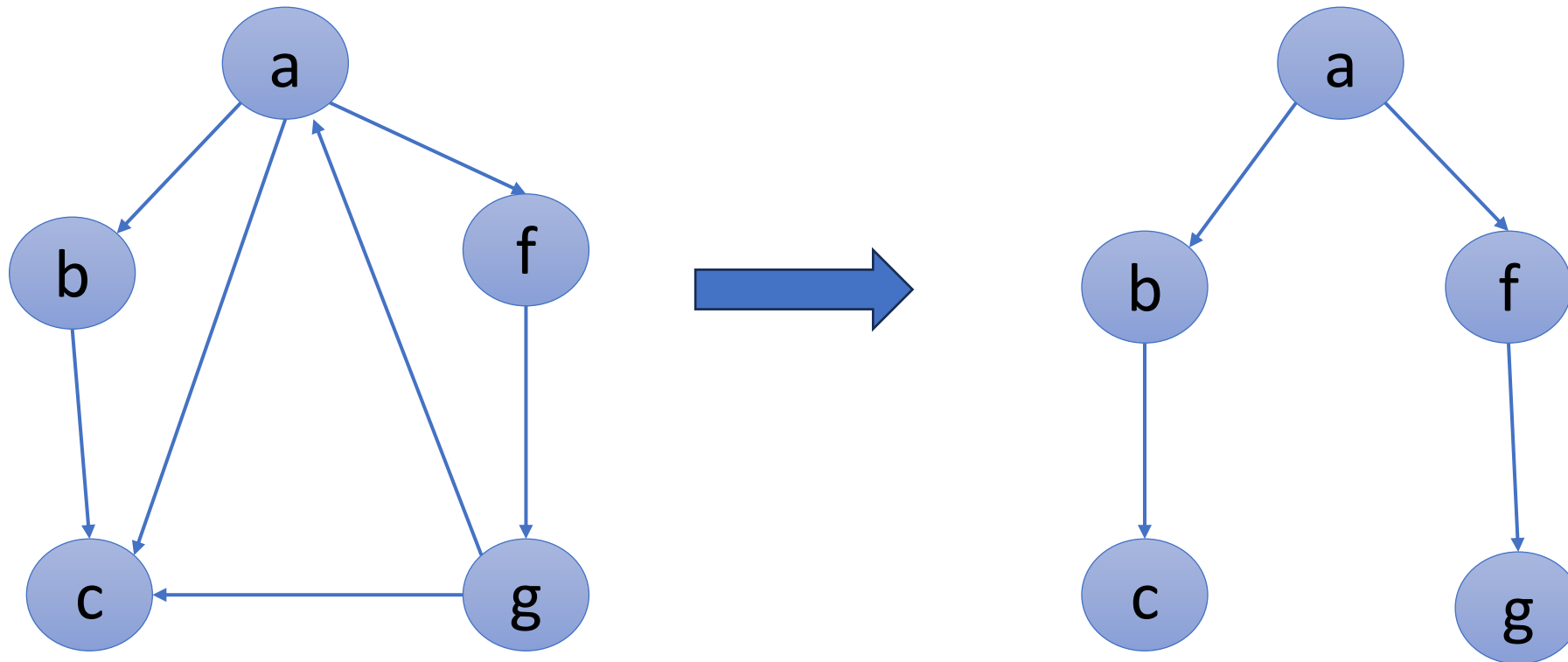


|         |  |   |   |   |   |   |   |   |   |   |    |    |
|---------|--|---|---|---|---|---|---|---|---|---|----|----|
| Stack:  |  |   |   |   |   |   |   |   |   |   |    |    |
|         |  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| Visited |  | T | T | T | T | T | T | T | T | T | T  | T  |

- Depth First Search (DFS)
- **DFS tree and và structure Num, Low**
- Find bridges (cạnh cầu)
- Find articulation vertex (đỉnh khớp)
- Find strongly connected component (thành phần liên thông mạnh)

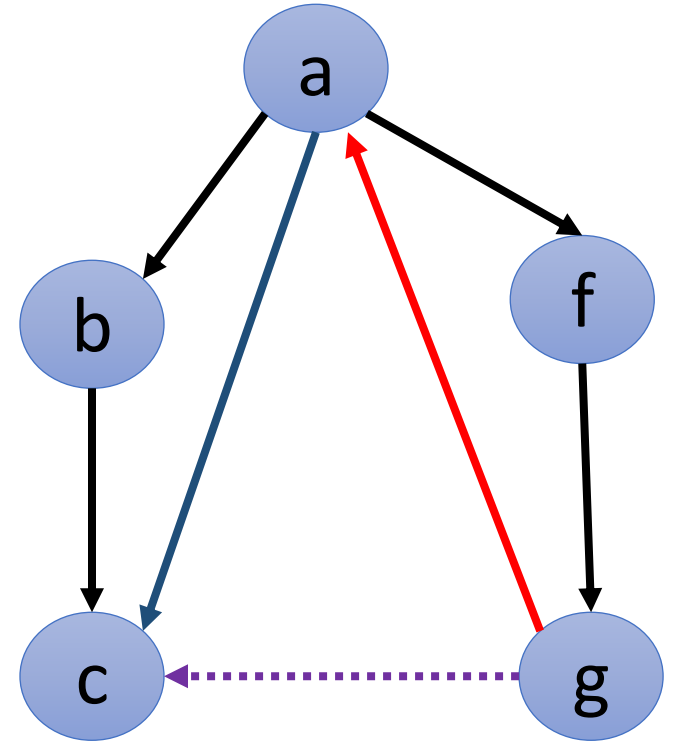
# DFS tree

- The trace of the depth-first search will form a tree



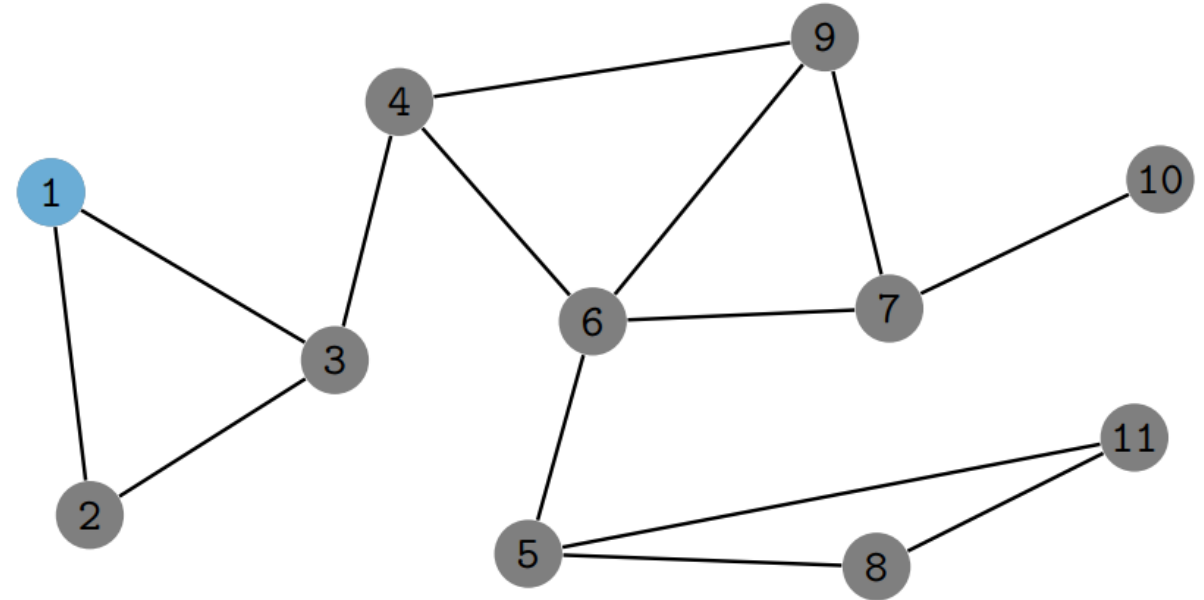
# DFS tree

- The trace of the depth-first search will form a tree
- Some kind of edge in the search process:
  - Tree Edge (Cạnh cây): The edge along which a new vertex is visited from one vertex, for example the black edge in the figure
  - Back Edge (Cạnh ngược): The edge going from descendant to ancestor, for example the red edge  $(g,a)$  in the figure
  - Forward Edge (Cạnh xuôi): The edge going from ancestors to descendants, for example the blue edge  $(a,c)$  in the figure
  - Crossing Edge (Cạnh vòng): The edge connecting two unrelated vertices, for example the dashed purple edge  $(c,g)$  in the figure



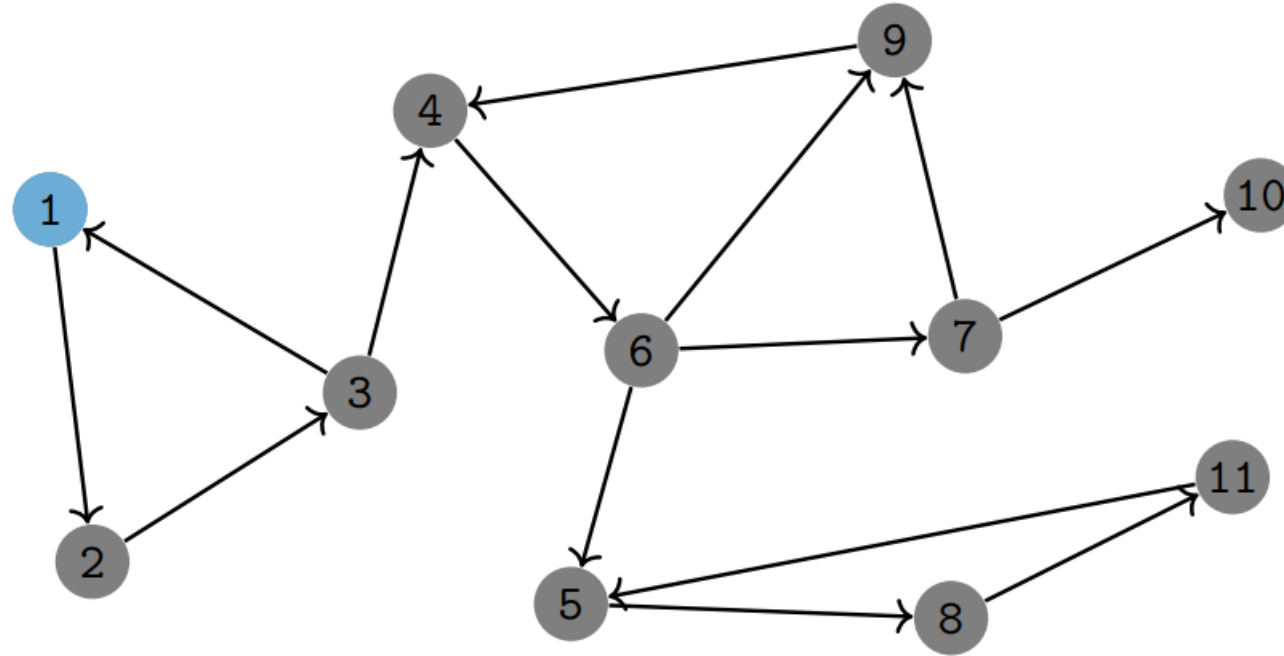
# DFS tree

- Arrays *Num* and *Low* store information of vertices in DFS tree:
  - Num*[*u*]: visit order of vertex *u* in DFS
  - Low*[*u*]: has the smallest value among the following values:
    - Num*[*v*] if (*v*, *u*) is a back edge
    - Low*[*v*] if *v* is a child of *u* in DFS tree
    - Num*[*u*]



| i      | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9  | 10 | 11 |
|--------|---|---|---|---|---|---|---|---|----|----|----|
| Num[i] | 1 | 2 | 3 | 4 | 6 | 5 | 9 | 7 | 10 | 11 | 8  |
| Low[i] | 1 | 1 | 1 | 4 | 6 | 4 | 4 | 6 | 4  | 11 | 6  |

# Example



| i      | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9  | 10 | 11 |
|--------|---|---|---|---|---|---|---|---|----|----|----|
| Num[i] | 1 | 2 | 3 | 4 | 6 | 5 | 9 | 7 | 10 | 11 | 8  |
| Low[i] | 1 | 1 | 1 | 4 | 6 | 4 | 4 | 6 | 4  | 11 | 6  |

# Implementation ideas

$cur\_num = 0$

**for** each  $u$  in  $V$ ,  $Num[u] = -1$

**Analyze**( $v, pv$ )

$Low[v] = Num[v] = cur\_num++$

**for** each  $u$  in  $Adj[v]$

**if**  $u == pv$  **continue**

**if**  $Num[u] == -1$

**Analyze**( $u, v$ )

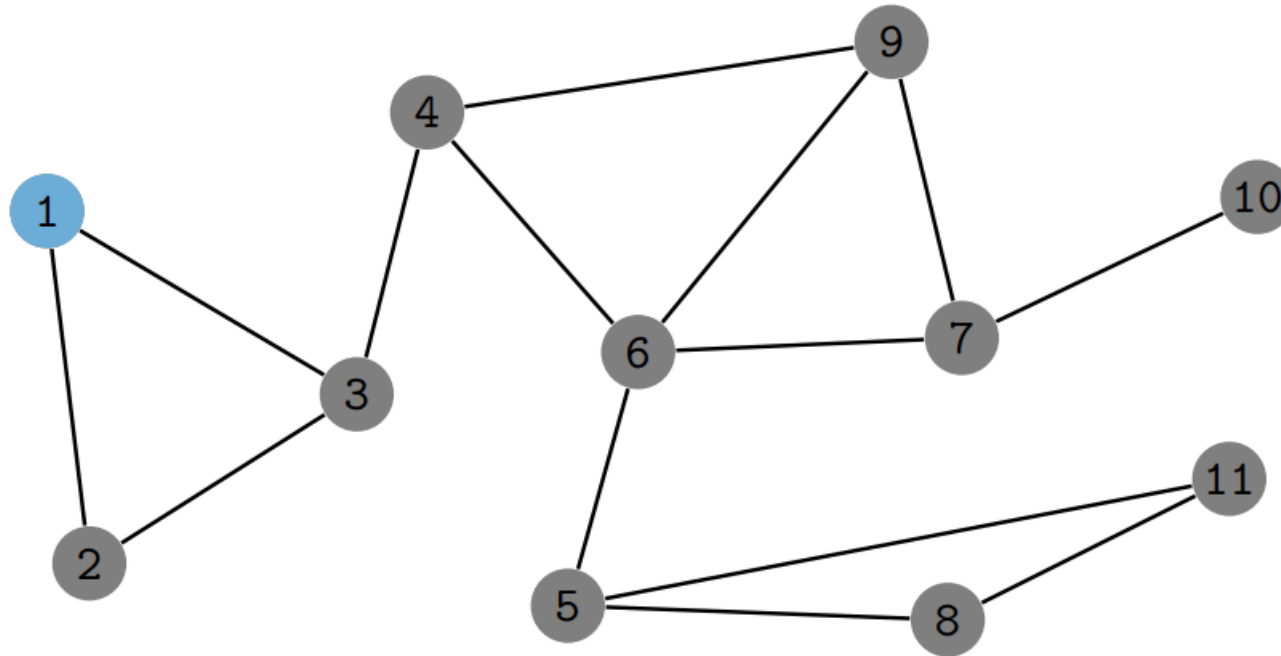
$Low[u] = \min(Low[u], Low[v])$

**else**  $Low[u] = \min(Low[u], Low[v])$

- Depth First Search (DFS)
- DFS tree and và structure Num, Low
- **Find bridges (cạnh cầu)**
- Find articulation vertex (đỉnh khớp)
- Find strongly connected component (thành phần liên thông mạnh)

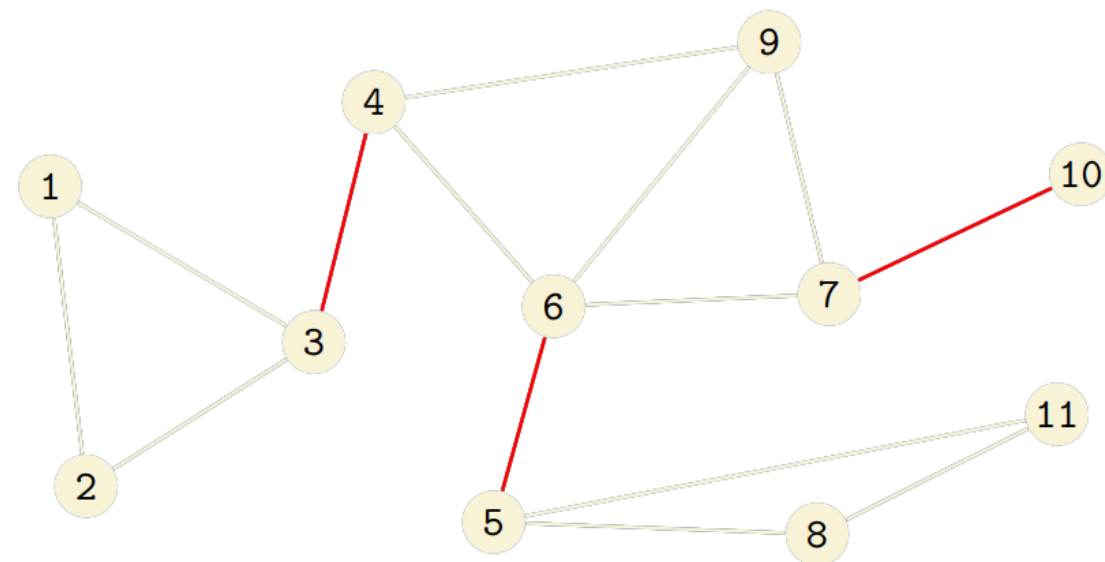
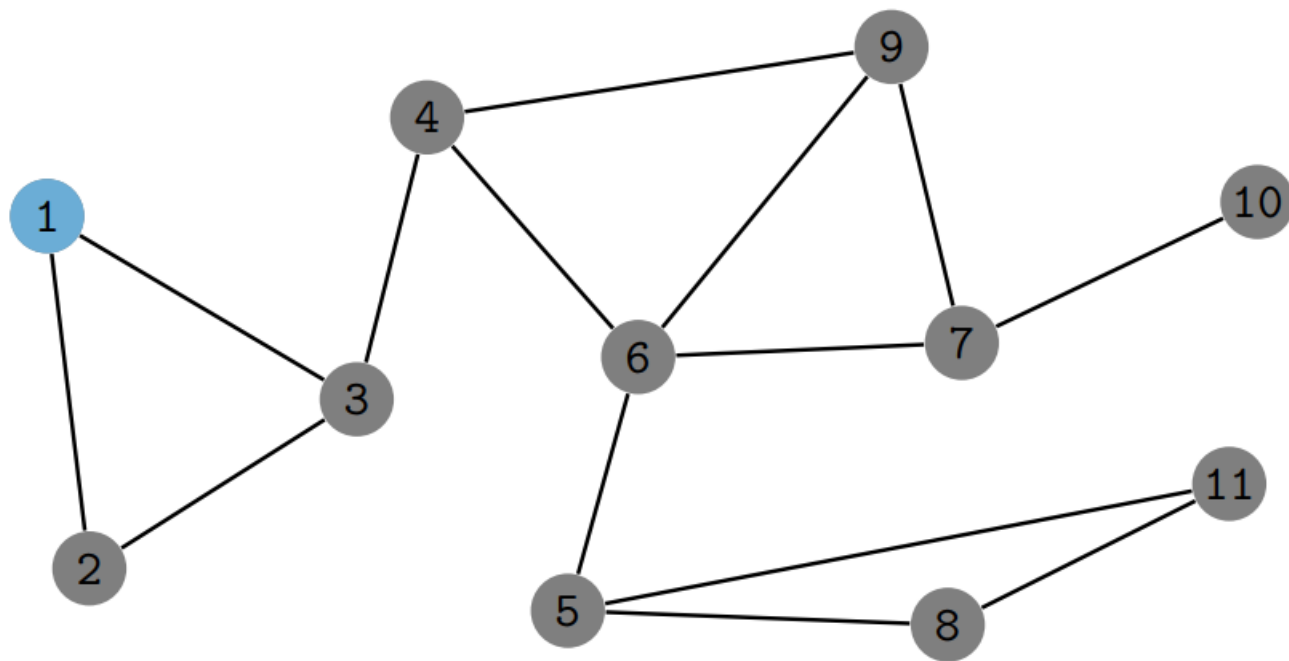
# Find bridge (cầu) in the graph

- **Definition:** A bridge is an edge of an undirected graph, so that removing this edge from the graph will increase the number of connected components.
- **Comment:** A forward edge  $(u, v)$  is bridge if and only if  $Low[v] > Num[u]$



# Find bridge (cầu) in the graph

- **Definition:** A bridge is an edge of an undirected graph, so that removing this edge from the graph will increase the number of connected components.
- **Comment:** A forward edge  $(u, v)$  is bridge if and only if  $Low[v] > Num[u]$



| i      | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9  | 10 | 11 |
|--------|---|---|---|---|---|---|---|---|----|----|----|
| Num[i] | 1 | 2 | 3 | 4 | 6 | 5 | 9 | 7 | 10 | 11 | 8  |
| Low[i] | 1 | 1 | 1 | 4 | 6 | 4 | 4 | 6 | 4  | 11 | 6  |

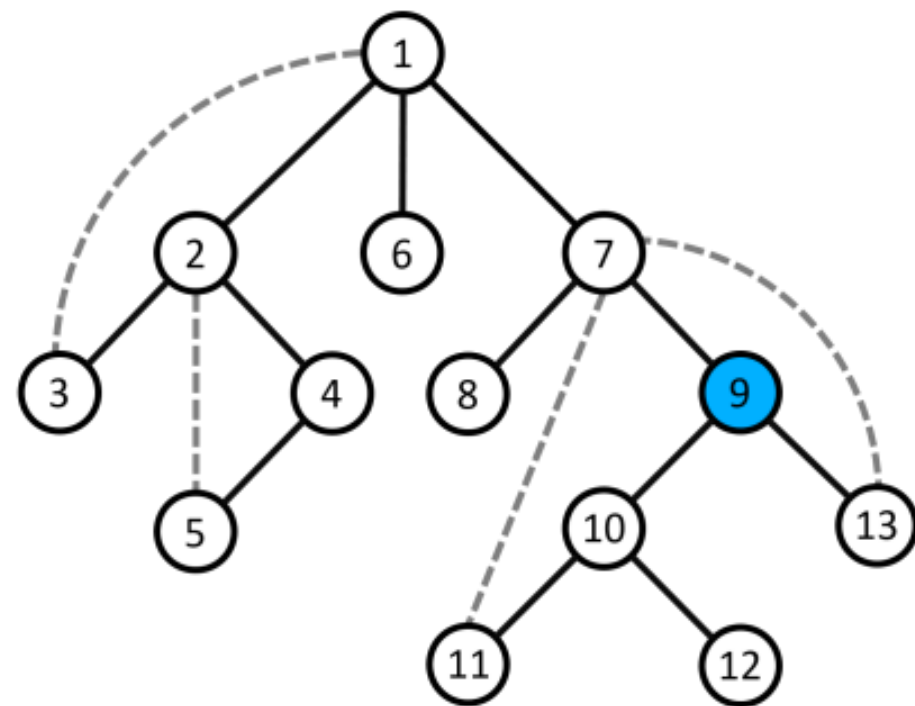
# Implementation idea

```
cur_num = 0
for each u in V, Num[u] = -1
Vector<pair<int, int>> bridgeLst;
findBridges(v, pv)
    Low[v] = Num[v] = cur_num + +
    for each u in Adj[v]
        if u == pv continue
        if Num[u] == -1
            findBridges(u, v)
            Low[u] = min(Low[u], Low[v])
        else Low[u] = min(Low[u], Low[v])
        if Low[v] > Num[u]
            bridgeLst.push(make_pair(u, v))
```

- Depth First Search (DFS)
- DFS tree and và structure Num, Low
- Find bridges (cạnh cầu)
- **Find articulation vertex (đỉnh khớp)**
- Find strongly connected component (thành phần liên thông mạnh)

# Find articulation vertex (đỉnh khớp)

- **Definition:** In an undirected graph, a vertex is called an articulation vertex if removing this vertex and edges having it as end point from the graph will increase the number of connected components of the graph.
- **Comment:** Vertex  $u$  vertex is called an articulation vertex if:
  - Either vertex  $u$  is not the root of DFS tree and  $Low[v] > Num[u]$  (where  $v$  is any direct child of  $u$  in the DFS tree);
  - Or vertex  $u$  is the root of the DFS tree and has at least 2 direct children.



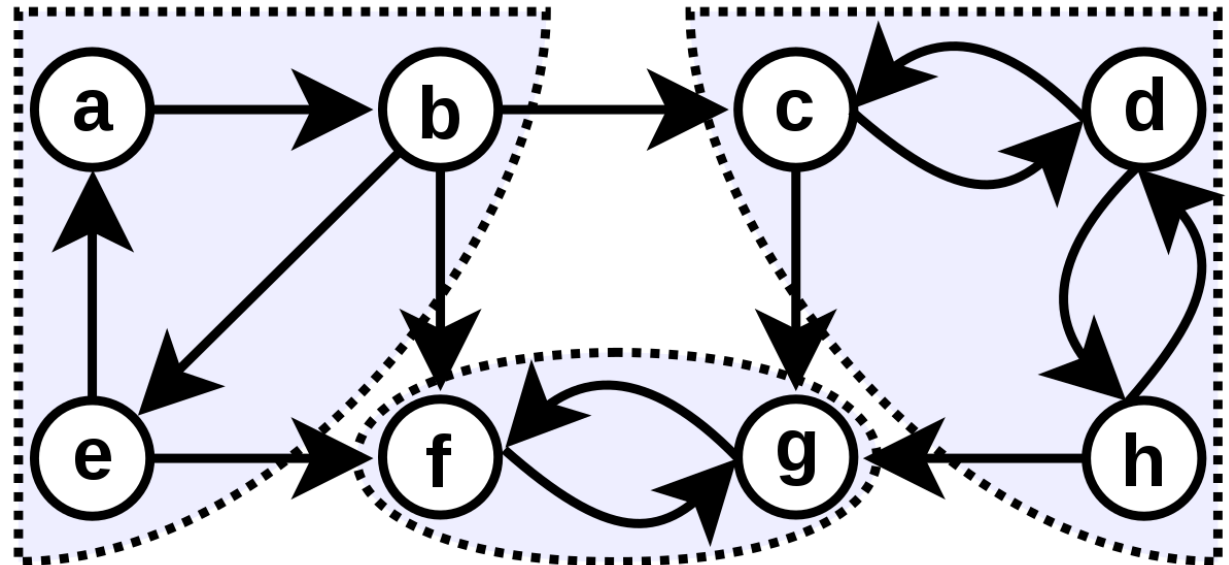
# Implementation idea

```
cur_num = 0
for each  $u$  in  $V$ ,  $Num[u] = -1$ 
bool joint[MAX];
findArticulation ( $v, pv$ )
    Child = 0
    Low[v] = Num[v] = cur_num++
    for each  $u$  in Adj[v]
        if  $u == pv$  continue
        if  $Num[u] == -1$ 
            findArticulation( $u, v$ )
            child++
            Low[u] = min(Low[u], Low[v])
            if  $u == pv$  and  $child > 1$ 
                joint[u] = true
            else if  $Low \geq Num[u]$ 
                joint[u] = true
        else Low[u] = min(Low[u], Low[v])
```

- Depth First Search (DFS)
- DFS tree and và structure Num, Low
- Find bridges (cạnh cầu)
- Find articulation vertex (đỉnh khớp)
- **Find strongly connected component (thành phần liên thông mạnh)**

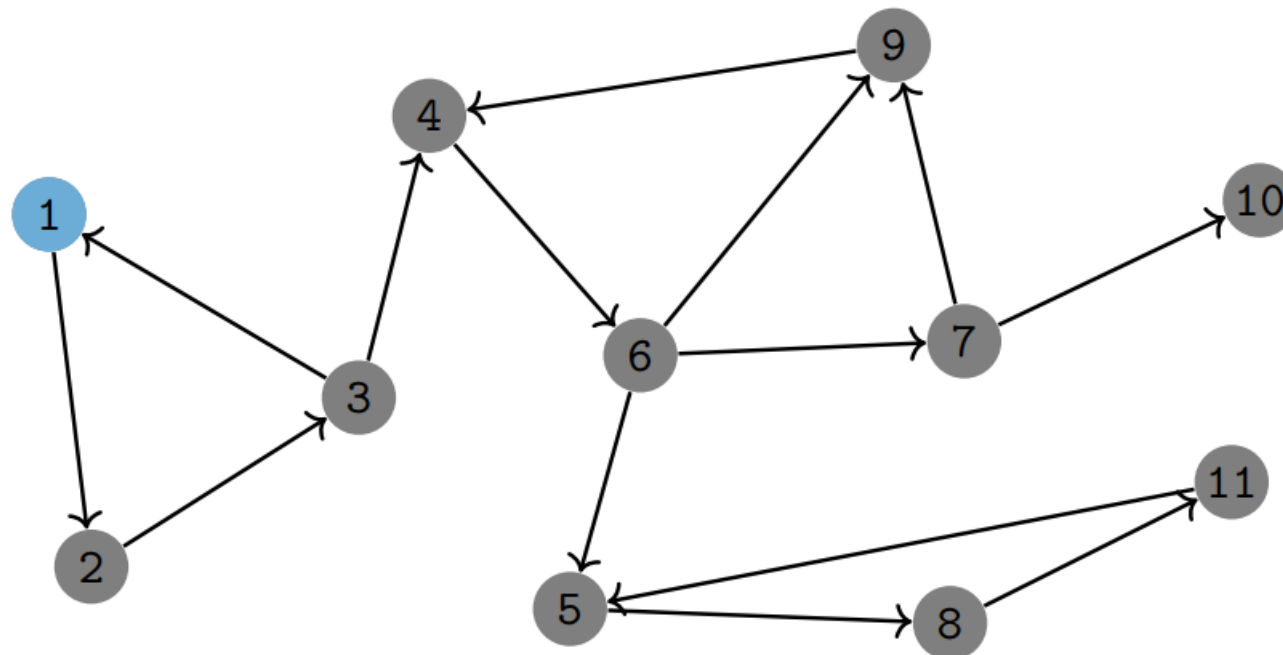
# Find Strongly Connected Components

- Breadth-First-Search (BFS) and Depth-First-Search (DFS) can easily find all connected components in an undirected graph. However, finding all strongly connected components is not simple for directed graph.
  - Note: A strongly connected component is a maximum subset of vertices such that between any two vertices there is always a path from one vertex to the other and vice versa.
- DFS search trees can be used to find
- all strongly connected components?



# Thuật toán Tarjan

- **Comment:** After analyzing the DFS tree, if at a vertex  $u$ ,  $Num[u] = Low[u]$ , then we have a strongly connected component following the process of traversing the tree from  $u$ .
- Use Stack to list vertices in a strongly connected component.
- Complexity:  $O(|V| + |E|)$



| i      | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9  | 10 | 11 |
|--------|---|---|---|---|---|---|---|---|----|----|----|
| Num[i] | 1 | 2 | 3 | 4 | 6 | 5 | 9 | 7 | 10 | 11 | 8  |
| Low[i] | 1 | 1 | 1 | 4 | 6 | 4 | 4 | 6 | 4  | 11 | 6  |

# Implementation idea

$cur\_num = 0$

for each  $u$  in  $V$ ,  $Num[u] = -1$

Stack  $S$

**SCC**( $v, pv$ )

$Low[v] = Num[v] = cur\_num++$

$S.push(v)$

for each  $u$  in  $Adj[v]$

if  $u == pv$  continue

if  $Num[u] == -1$

**SCC**( $u, v$ )

$Low[u] = \min(Low[u], Low[v])$

else  $Low[u] = \min(Low[u], Low[v])$

if  $Num[u] == Low[u]$

while(true)

$v = S.top(), S.pop()$

cout <<  $v$  << " ";

if  $v == u$

break

A large graphic on the left side of the slide. It features a dark blue background with a circular pattern of red dots of varying sizes, creating a sense of depth and movement. The word "HUST" is centered within this graphic in a bold, white, sans-serif font.

# HUST

# THANK YOU !